

# A3\_comments\_interest

September 15, 2026

## 1 Assignment 3 - E-commerce Comments / Customer Interest Discovery

### 1.1 Intelligent System Development

#### 1.1.1 Dataset

eBay Reviews — `ebay_reviews.csv`

#### 1.1.2 Four models to compare

1. Logistic Regression
2. Linear SVM
3. Multinomial Naive Bayes
4. Deep Learning - NumPy from Scratch

The main system pipeline is:

```
Comment
  ↓
Tokens
  ↓
Token IDs
  ↓
Vector / Embedding
```

The Deep Learning section follows the agreed nine-step sequence exactly:

- Step 1 - Implement Activation Functions
- Step 2 - Initialize the Network
- Step 3 - Forward Propagation
- Step 4 - Cross-Entropy Loss
- Step 5 - Backpropagation
- Step 6 - Gradient Descent
- Step 7 - Training Loop
- Step 8 - Prediction
- Step 9 - Evaluation

**Important restriction:** the neural network is implemented with **NumPy only**. No TensorFlow, PyTorch, Keras, or other Deep Learning framework is used.

## 1.2 1. Learning Objectives

By the end of this notebook we should be able to:

1. Understand the structure, meaning, and limitations of the eBay reviews dataset.
2. Audit and clean missing values, duplicates, invalid values, and review text.
3. Define a concrete e-commerce customer-interest task from the available fields.
4. Convert natural language reviews into a mathematical representation for machine learning.
5. Perform feature engineering and feature selection so that the model uses informative features and avoids unnecessary dimensions.
6. Train three traditional machine-learning classifiers.
7. Build a text neural network from scratch using NumPy and a learnable embedding layer.
8. Evaluate and compare **all four models together on the same held-out test set**.
9. Persist the best ML model and the Deep Learning model for Web/Mobile inference.

## 1.3 2. Problem Definition - From Reviews to Product Interest

the e-commerce application may use customer comments/reviews to discover interests such as the product category a customer is interested in.

For this dataset, one row represents one **product review**. The available attributes are:

- `category` - product category associated with the review
- `review title` - short review title
- `review content` - review text
- `rating` - numerical review rating from 1 to 5

We define the supervised task as:

`X = review title + review content + auxiliary rating signal`  
`y = product-category interest`

The raw `category` field contains a long tail with many categories that have only a few observations. For a robust multi-class experiment and a from-scratch NumPy neural network, we define the target as the **15 most frequent product categories plus one Other class**.

This is a task-definition choice, not a claim that the rare categories are unimportant. The raw category distribution is still reported in the EDA section.

### 1.3.1 Main question

Can the review text, supported by the rating signal, predict the product category that represents the reviewer's product interest?

This makes the application a **multi-class text classification** problem.

### 1.3.2 Dataset Source

The uploaded file is the raw `ebay_reviews.csv` dataset described on Kaggle as an eBay review dataset with four columns: product category, review title, review content, and rating.

Kaggle source:

<https://www.kaggle.com/datasets/wojtekbbonicki/ebay-reviews>

The Kaggle page describes `ebay_reviews.csv` as the uncleaned version that should be preprocessed before machine learning.

## 2 3. Imports and Reproducibility

The traditional ML part may use scikit-learn because the A3 requirement only forbids external Deep Learning frameworks for the neural network.

The Deep Learning section itself uses:

- NumPy arrays
- matrix multiplication
- ReLU
- Softmax
- cross-entropy
- manual backpropagation
- gradient descent

No TensorFlow / PyTorch / Keras is imported or used.

```
[1]: import os
import re
import json
import time
import random
import warnings
from pathlib import Path
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.sparse import hstack, csr_matrix
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
```

```

        confusion_matrix,
        classification_report,
        roc_auc_score,
        roc_curve,
    )

import joblib

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)
random.seed(SEED)

NOTEBOOK_DIR = Path.cwd()
DATA_PATH = Path("/mnt/data/ebay_reviews.csv") if Path("/mnt/data/ebay_reviews.
↪ csv").exists() else NOTEBOOK_DIR / "data/ebay_reviews.csv"

MODEL_DIR = NOTEBOOK_DIR / "model_comments"
FIG_DIR = MODEL_DIR / "figures"
MODEL_DIR.mkdir(parents=True, exist_ok=True)
FIG_DIR.mkdir(parents=True, exist_ok=True)

print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
print("scikit-learn version:", __import__("sklearn").__version__)
print("Random seed:", SEED)
print("Data path:", DATA_PATH)

```

```

NumPy version: 2.4.6
Pandas version: 3.0.3
scikit-learn version: 1.9.0
Random seed: 42
Data path: C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\comments\data\ebay_reviews.csv

```

### 3 4. Locate and Load the Dataset

The raw CSV is loaded first and kept unchanged.

```

[2]: if not DATA_PATH.exists():
        raise FileNotFoundError(
            f"Cannot find {DATA_PATH}. Place ebay_reviews.csv in the data folder "
            "or update DATA_PATH."
        )

df_raw = pd.read_csv(DATA_PATH)

```

```

print("First 5 rows:")
display(df_raw.head())

print("\nShape:", df_raw.shape)
print("\nColumn names:")
print(df_raw.columns.tolist())

print("\nData types:")
display(df_raw.dtypes.to_frame("dtype"))

```

First 5 rows:

|   | category | review title \                                   |
|---|----------|--------------------------------------------------|
| 0 | Headsets | Wireless gaming headset                          |
| 1 | Headsets | Good for those with a big head, low budget       |
| 2 | Headsets | MezumiWireless Gaming Headset                    |
| 3 | Headsets | HW- S2 great headset.                            |
| 4 | Headsets | BEST HEADPHONES I'VE PURCHASED IN MY ENTIRE LIFE |

|   | review content                                    | rating |
|---|---------------------------------------------------|--------|
| 0 | This gaming headset ticks all the boxes # look... | 5      |
| 1 | Easy setup, rated for 6 hours battery but mine... | 3      |
| 2 | I originally bought this wireless headset for ... | 5      |
| 3 | This is my 2nd Mezumi headset, It kills the fi... | 5      |
| 4 | This is probably the best headset I've purchas... | 5      |

Shape: (44756, 4)

Column names:

```
['category', 'review title', 'review content', 'rating']
```

Data types:

|                | dtype |
|----------------|-------|
| category       | str   |
| review title   | str   |
| review content | str   |
| rating         | int64 |

### 3.1 4.1 Initial Dataset Observations

The current uploaded dataset contains **44,756 observations and 4 attributes**.

One observation is one product review. The two text columns are the core inputs for the Comments / Interest problem.

The raw table is already small enough to inspect directly, but the text representation will create a much higher-dimensional feature space than the original CSV.

```
[3]: print("Rows:", len(df_raw))
      print("Columns:", df_raw.shape[1])

      observation_table = pd.DataFrame({
          "Column": df_raw.columns,
          "Data type": [str(df_raw[c].dtype) for c in df_raw.columns],
          "Missing": [int(df_raw[c].isna().sum()) for c in df_raw.columns],
          "Unique values": [int(df_raw[c].nunique(dropna=True)) for c in df_raw.
          ↪columns],
      })
      display(observation_table)

      print("\nExample observation:")
      display(df_raw.iloc[[0]])
```

Rows: 44756

Columns: 4

|   | Column         | Data type | Missing | Unique values |
|---|----------------|-----------|---------|---------------|
| 0 | category       | str       | 0       | 162           |
| 1 | review title   | str       | 2       | 31899         |
| 2 | review content | str       | 0       | 42293         |
| 3 | rating         | int64     | 0       | 5             |

Example observation:

|   | category | review title            | review content                                    | rating |
|---|----------|-------------------------|---------------------------------------------------|--------|
| 0 | Headsets | Wireless gaming headset | This gaming headset ticks all the boxes # look... | 5      |

## 4 5. Data Quality Audit

A professional cleaning workflow should answer these questions explicitly:

1. Are there missing values?
2. Are there exact duplicate rows?
3. Are ratings valid?
4. Are categories blank or malformed?
5. Are review title/content fields empty?
6. Does the text contain boilerplate that should not become model features?
7. Are there extreme text lengths that need monitoring?

Each issue is checked separately before the modeling table is created.

### 4.0.1 5.1 Missing Values

The original dataset contains two missing values in `review title` and no missing values in `review content`, `category`, or `rating`.

**Decision:** - `review title` is optional auxiliary text, so missing titles are filled with an empty string. - `review content` is the essential text field. If it were missing, a non-empty title could be used as a fallback; rows with neither title nor content would be removed. - `category` and `rating` are required for supervised training; invalid or missing targets cannot be imputed.

```
[4]: missing_summary = (
      df_raw.isna()
      .sum()
      .sort_values(ascending=False)
      .to_frame("missing_count")
    )
    missing_summary["missing_percent"] = (
        missing_summary["missing_count"] / len(df_raw) * 100
    ).round(3)

    display(missing_summary)

    print("Decision:")
    print("- Missing review title -> fill with empty string.")
    print("- Missing review content -> use title as fallback; drop only if both are empty.")
    print("- Missing category/rating -> cannot create a valid supervised target.")
```

|                | missing_count | missing_percent |
|----------------|---------------|-----------------|
| review title   | 2             | 0.004           |
| category       | 0             | 0.000           |
| review content | 0             | 0.000           |
| rating         | 0             | 0.000           |

Decision:

- Missing review title -> fill with empty string.
- Missing review content -> use title as fallback; drop only if both are empty.
- Missing category/rating -> cannot create a valid supervised target.

### 4.0.2 5.2 Duplicate Rows

Exact duplicate rows represent repeated complete observations.

**Decision:** remove exact duplicates before splitting the data, because otherwise the same review could receive unintended extra weight in training.

Only exact row duplicates are removed. Similar but non-identical reviews are not removed.

```
[5]: duplicate_count = int(df_raw.duplicated().sum())
    print("Exact duplicate rows:", duplicate_count)
```

```

if duplicate_count > 0:
    print("Action: exact duplicates will be removed.")
else:
    print("Action: no exact duplicate rows are present, so no duplicate removal_
↪is required.")

```

Exact duplicate rows: 0

Action: no exact duplicate rows are present, so no duplicate removal is required.

### 4.0.3 5.3 Rating and Category Validity

Ratings should be integer values from 1 to 5.

The category is the supervised target, so blank categories cannot define a valid class.

```

[6]: rating_numeric = pd.to_numeric(df_raw["rating"], errors="coerce")
invalid_rating_mask = rating_numeric.isna() | ~rating_numeric.isin([1, 2, 3, 4,
↪5])

blank_category_mask = (
    df_raw["category"].isna()
    | (df_raw["category"].astype(str).str.strip() == "")
)

print("Invalid rating rows:", int(invalid_rating_mask.sum()))
print("Blank category rows:", int(blank_category_mask.sum()))

print("\nObserved rating values:")
print(sorted(pd.Series(rating_numeric.dropna().unique()).tolist()))

```

Invalid rating rows: 0

Blank category rows: 0

Observed rating values:

[1, 2, 3, 4, 5]

### 4.0.4 5.4 Empty Text and Text Boilerplate

A title may legitimately be empty, but the combined review representation must contain actual text. We therefore inspect the number of empty titles and contents.

The source data also contains the phrase `Read full review...` in some review contents. This phrase is web-page boilerplate rather than customer language, so it is removed during text normalization.

```

[7]: title_empty = df_raw["review title"].fillna("").astype(str).str.strip().eq("").
↪sum()
content_empty = df_raw["review content"].fillna("").astype(str).str.strip().
↪eq("").sum()

```

```

print("Empty review titles:", int(title_empty))
print("Empty review contents:", int(content_empty))

boilerplate_count = (
    df_raw["review content"].fillna("").astype(str)
    .str.contains(r"Read full review\\.\\.\\.\"", case=False, regex=True)
    .sum()
)

print("Rows containing 'Read full review...':", int(boilerplate_count))

```

```

Empty review titles: 2
Empty review contents: 0
Rows containing 'Read full review...': 12436

```

#### 4.0.5 5.5 Cleaning Summary

The cleaning policy used below is explicit:

1. Copy the raw data.
2. Drop exact duplicate rows.
3. Coerce `rating` to numeric and keep only values in 1–5.
4. Strip whitespace from `category`.
5. Fill missing `review title` with "".
6. Use the review title as a fallback only when review content is missing.
7. Drop rows where both text fields are empty.
8. Normalize text by lowercasing, removing the known `Read full review...` boilerplate, normalizing whitespace, and retaining word characters needed by the tokenizer.

No arbitrary statistical outlier deletion is performed on review length. Very long reviews are legitimate observations; they are handled later by a maximum sequence length only in the Deep Learning representation.

```

[8]: df = df_raw.copy()

# 1) Exact duplicate removal.
df = df.drop_duplicates().reset_index(drop=True)

# 2) Clean target and rating.
df["category"] = df["category"].fillna("").astype(str).str.strip()
df["rating"] = pd.to_numeric(df["rating"], errors="coerce")

# Keep only valid ratings and non-empty category.
df = df[df["rating"].isin([1, 2, 3, 4, 5])].copy()
df = df[df["category"].ne("")] .copy()

# 3) Missing title/content handling.
df["review title"] = df["review title"].fillna("").astype(str)

```

```

df["review content"] = df["review content"].fillna("").astype(str)

missing_content = df["review content"].str.strip().eq("")
fallback_mask = missing_content & df["review title"].str.strip().ne("")
df.loc[fallback_mask, "review content"] = df.loc[fallback_mask, "review title"]

# Drop rows with no usable text at all.
no_text_mask = (
    df["review title"].str.strip().eq("")
    & df["review content"].str.strip().eq("")
)
df = df.loc[~no_text_mask].copy()

# Keep rating as an integer auxiliary signal.
df["rating"] = df["rating"].astype(int)

print("Rows after cleaning:", len(df))
print("Rows removed:", len(df_raw) - len(df))
print("Missing values after cleaning:")
display(df[["category", "review title", "review content", "rating"]].isna().
    ↪sum().to_frame("missing"))

```

Rows after cleaning: 44756

Rows removed: 0

Missing values after cleaning:

|                | missing |
|----------------|---------|
| category       | 0       |
| review title   | 0       |
| review content | 0       |
| rating         | 0       |

```

[9]: def normalize_review_text(text):
    text = str(text).lower()
    text = re.sub(r"read full review\\.\\.\\. ", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"<[>]+>", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

df["title_clean"] = df["review title"].map(normalize_review_text)
df["content_clean"] = df["review content"].map(normalize_review_text)

# Title + content are the main review representation.
df["review_text"] = (
    df["title_clean"].str.strip()
    + " [SEP] "
    + df["content_clean"].str.strip()
).str.strip()

```

```
print("Example cleaned review:")
display(
    df[["review title", "review content", "review_text"]].head(3)
)

print("\nRemaining empty combined texts:", int(df["review_text"].str.strip().
    ↪eq("").sum()))
```

Example cleaned review:

```

                                review title \
0                Wireless gaming headset
1  Good for those with a big head, low budget
2                MezumiWireless Gaming Headset

                                review content \
0  This gaming headset ticks all the boxes # look...
1  Easy setup, rated for 6 hours battery but mine...
2  I originally bought this wireless headset for ...

                                review_text
0  wireless gaming headset [SEP] this gaming head...
1  good for those with a big head, low budget [SE...
2  mezumiwireless gaming headset [SEP] i original...
```

Remaining empty combined texts: 0

## 5 6. Exploratory Data Analysis

Before choosing the representation, we inspect:

1. Target/category distribution
2. Rating distribution
3. Review length distribution
4. Title and content length
5. Category coverage and the long-tail problem
6. Relationship between ratings and the most common categories

The purpose is not to pick a model from the plots. The purpose is to understand the data and justify preprocessing and feature-engineering decisions.

```
[10]: category_counts = df["category"].value_counts()

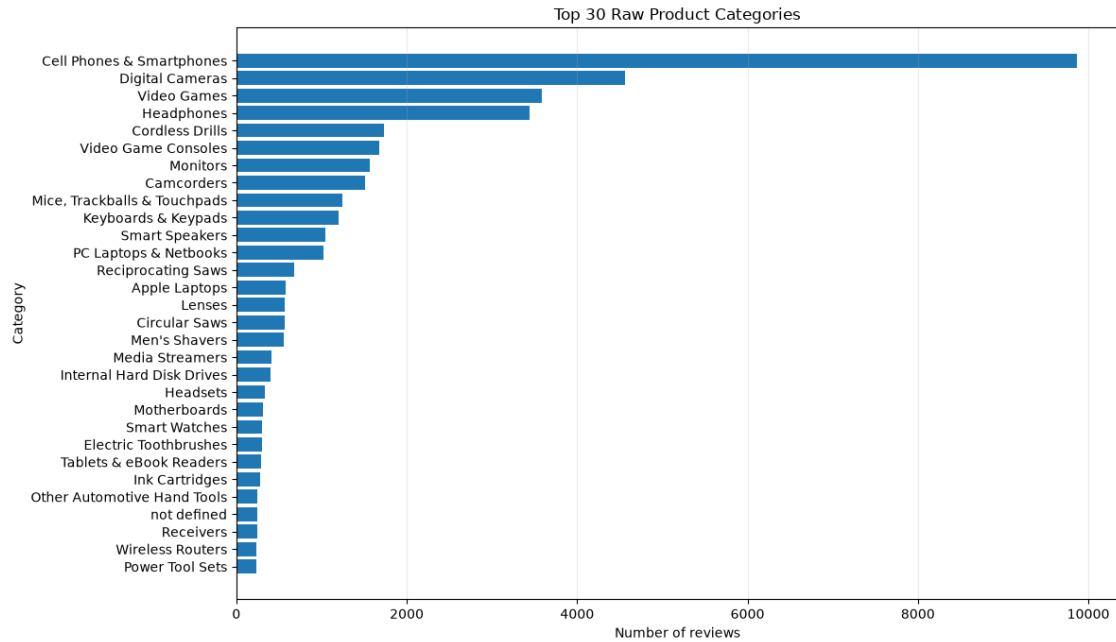
print("Number of unique raw categories:", len(category_counts))
print("Top 20 categories:")
display(category_counts.head(30).to_frame("count"))
```

Number of unique raw categories: 162

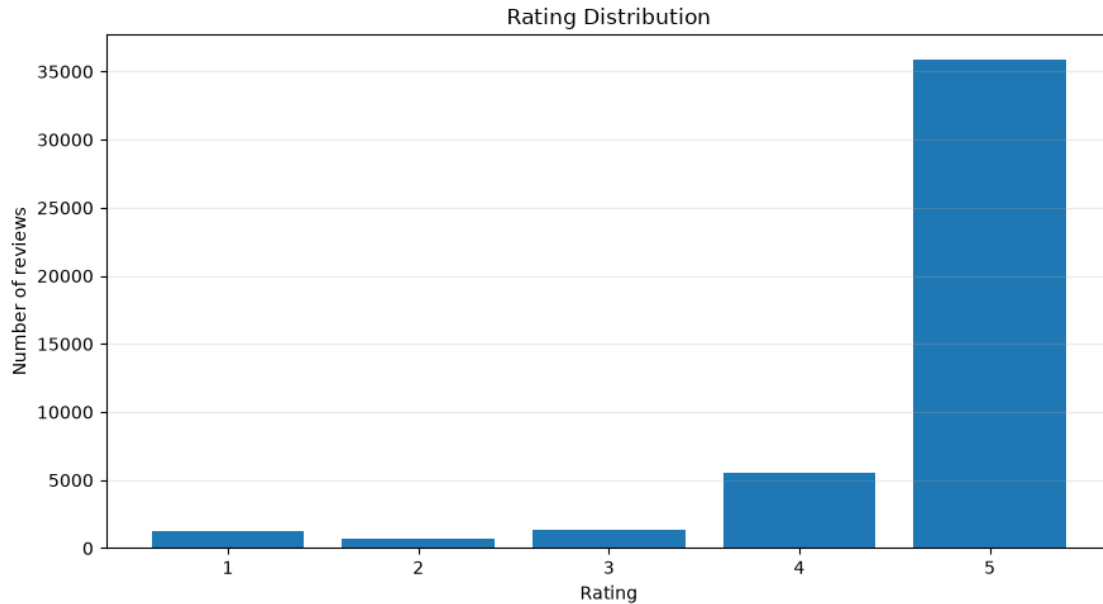
Top 20 categories:

| category                     | count |
|------------------------------|-------|
| Cell Phones & Smartphones    | 9861  |
| Digital Cameras              | 4565  |
| Video Games                  | 3588  |
| Headphones                   | 3442  |
| Cordless Drills              | 1736  |
| Video Game Consoles          | 1672  |
| Monitors                     | 1563  |
| Camcorders                   | 1514  |
| Mice, Trackballs & Touchpads | 1247  |
| Keyboards & Keypads          | 1201  |
| Smart Speakers               | 1042  |
| PC Laptops & Netbooks        | 1018  |
| Reciprocating Saws           | 678   |
| Apple Laptops                | 583   |
| Lenses                       | 573   |
| Circular Saws                | 573   |
| Men's Shavers                | 557   |
| Media Streamers              | 413   |
| Internal Hard Disk Drives    | 400   |
| Headsets                     | 336   |
| Motherboards                 | 308   |
| Smart Watches                | 306   |
| Electric Toothbrushes        | 297   |
| Tablets & eBook Readers      | 295   |
| Ink Cartridges               | 284   |
| Other Automotive Hand Tools  | 251   |
| not defined                  | 246   |
| Receivers                    | 243   |
| Wireless Routers             | 232   |
| Power Tool Sets              | 231   |

```
[11]: plt.figure(figsize=(12, 7))
      top30 = category_counts.head(30).sort_values()
      plt.barh(top30.index, top30.values)
      plt.title("Top 30 Raw Product Categories")
      plt.xlabel("Number of reviews")
      plt.ylabel("Category")
      plt.grid(axis="x", alpha=0.25)
      plt.tight_layout()
      plt.savefig(FIG_DIR / "01_top30_categories.png", dpi=180)
      plt.show()
```



```
[12]: plt.figure(figsize=(9, 5))
rating_counts = df["rating"].value_counts().sort_index()
plt.bar(rating_counts.index.astype(str), rating_counts.values)
plt.title("Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Number of reviews")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.savefig(FIG_DIR / "02_rating_distribution.png", dpi=180)
plt.show()
```



```
[13]: df["title_char_len"] = df["title_clean"].str.len()
df["content_char_len"] = df["content_clean"].str.len()
df["review_word_count"] = df["review_text"].str.findall(r"\b\w+\b").str.len()

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].hist(df["title_char_len"], bins=40)
axes[0].set_title("Review Title Length")
axes[0].set_xlabel("Characters")
axes[0].set_ylabel("Count")

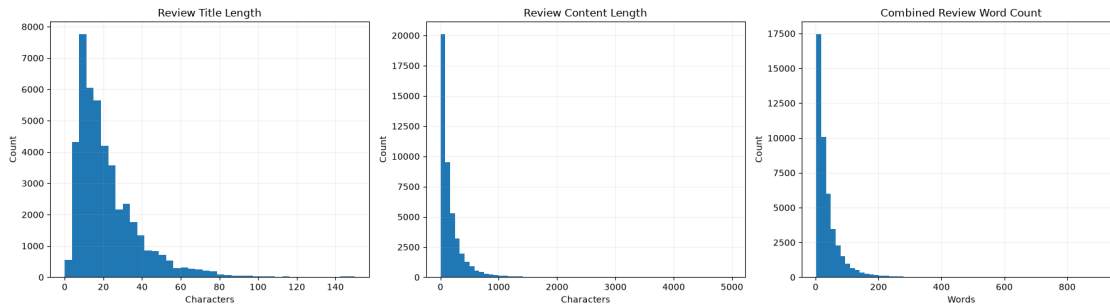
axes[1].hist(df["content_char_len"], bins=60)
axes[1].set_title("Review Content Length")
axes[1].set_xlabel("Characters")
axes[1].set_ylabel("Count")

axes[2].hist(df["review_word_count"], bins=60)
axes[2].set_title("Combined Review Word Count")
axes[2].set_xlabel("Words")
axes[2].set_ylabel("Count")

for ax in axes:
    ax.grid(alpha=0.2)

plt.tight_layout()
plt.savefig(FIG_DIR / "03_text_length_distributions.png", dpi=180)
plt.show()
```

```
display(
    df[["title_char_len", "content_char_len", "review_word_count"]]
    .describe(percentiles=[0.5, 0.75, 0.90, 0.95, 0.99])
    .round(2)
)
```



|       | title_char_len | content_char_len | review_word_count |
|-------|----------------|------------------|-------------------|
| count | 44756.00       | 44756.00         | 44756.00          |
| mean  | 22.23          | 186.67           | 40.28             |
| std   | 17.28          | 270.36           | 52.43             |
| min   | 0.00           | 1.00             | 2.00              |
| 50%   | 17.00          | 99.00            | 24.00             |
| 75%   | 28.00          | 224.00           | 49.00             |
| 90%   | 43.00          | 425.00           | 88.00             |
| 95%   | 55.00          | 618.00           | 126.00            |
| 99%   | 83.00          | 1287.00          | 254.00            |
| max   | 150.00         | 4974.00          | 924.00            |

```
[14]: # Long-tail coverage: how much of the dataset is represented by the top
      ↪ categories?
cum_coverage = category_counts.cumsum() / len(df)

coverage_points = {
    "Top 5": float(cum_coverage.iloc[min(4, len(cum_coverage)-1)]),
    "Top 10": float(cum_coverage.iloc[min(9, len(cum_coverage)-1)]),
    "Top 15": float(cum_coverage.iloc[min(14, len(cum_coverage)-1)]),
    "Top 20": float(cum_coverage.iloc[min(19, len(cum_coverage)-1)]),
}

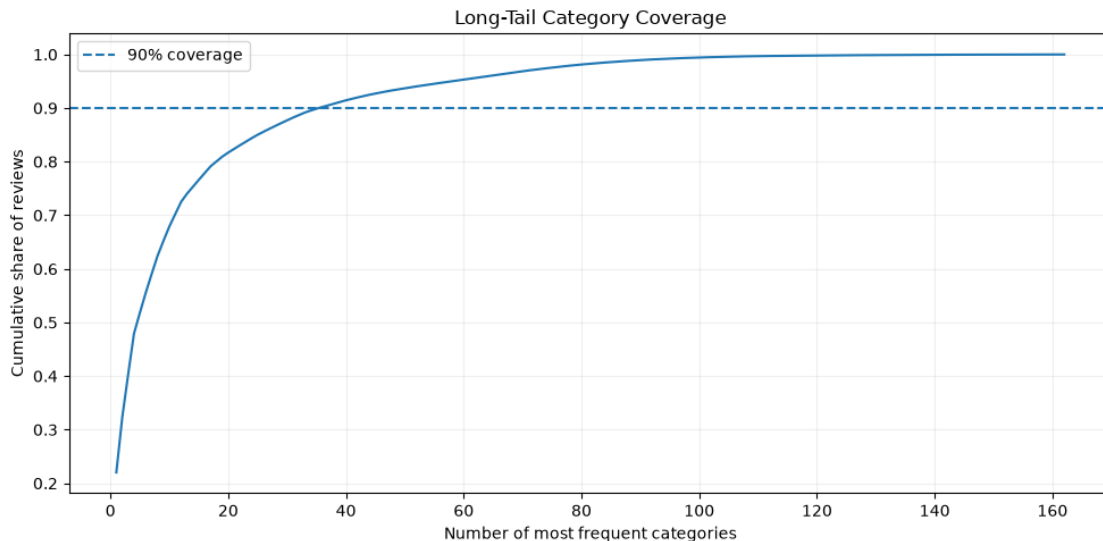
display(pd.DataFrame(
    {"Cumulative coverage": coverage_points}
).assign(**{"Cumulative coverage": lambda x: x["Cumulative coverage"]
    ↪round(3)}))
```

```

plt.figure(figsize=(10, 5))
plt.plot(np.arange(1, len(cum_coverage) + 1), cum_coverage.values)
plt.axhline(0.9, linestyle="--", label="90% coverage")
plt.xlabel("Number of most frequent categories")
plt.ylabel("Cumulative share of reviews")
plt.title("Long-Tail Category Coverage")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "04_category_coverage.png", dpi=180)
plt.show()

```

|        | Cumulative coverage |
|--------|---------------------|
| Top 5  | 0.518               |
| Top 10 | 0.679               |
| Top 15 | 0.766               |
| Top 20 | 0.817               |



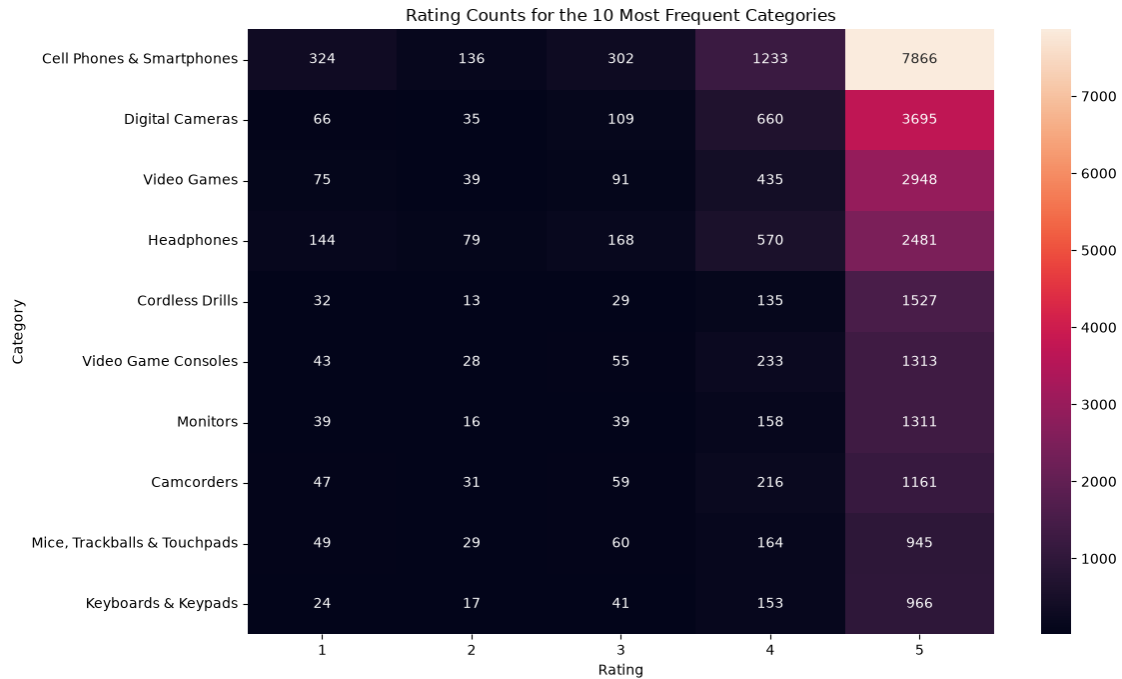
```

[15]: top10_categories = category_counts.head(10).index
rating_by_top_category = (
    df[df["category"].isin(top10_categories)]
    .groupby(["category", "rating"])
    .size()
    .unstack(fill_value=0)
    .reindex(top10_categories)
)

plt.figure(figsize=(12, 7))
sns.heatmap(rating_by_top_category, annot=True, fmt="d")

```

```
plt.title("Rating Counts for the 10 Most Frequent Categories")
plt.xlabel("Rating")
plt.ylabel("Category")
plt.tight_layout()
plt.savefig(FIG_DIR / "05_rating_by_top_category.png", dpi=180)
plt.show()
```



## 6 7. Feature Engineering and Target Definition

Feature engineering is especially important for this dataset because the original CSV contains only four columns, but not every field should be treated equally.

### 6.0.1 Feature decisions

**Keep** - review title: concise product/experience cues - review content: main interest-bearing text - rating: small auxiliary numerical signal

**Exclude** - category: this is the target, so using it as an input would be direct target leakage - any row index: it has no business meaning

### 6.0.2 Text representation for traditional ML

review title + review content

↓

TF-IDF word n-grams

↓

chi-square feature selection

↓

sparse numerical matrix

We use TF-IDF rather than feeding raw strings to the models. Feature selection is applied after TF-IDF so that low-information terms are removed.

### 6.0.3 Target engineering

The raw category space is highly long-tailed. We use the 15 most frequent raw categories as explicit classes and map every remaining category to `Other`.

The resulting task is therefore a 16-class classification problem.

```
[16]: TOP_K = 30
TOP_CATEGORIES = category_counts.head(TOP_K).index.tolist()

# For deployment/demo: use the top 30 real product-interest categories.
# Rare categories outside the top 30 are removed instead of merged into "Other".
df = df[df["category"].isin(TOP_CATEGORIES)].copy()

df["interest_category"] = df["category"]

CLASS_ORDER = TOP_CATEGORIES
CLASS_TO_ID = {name: i for i, name in enumerate(CLASS_ORDER)}
ID_TO_CLASS = {i: name for name, i in CLASS_TO_ID.items()}

df["target_id"] = df["interest_category"].map(CLASS_TO_ID).astype(int)

print("Final task classes:")
for i, cls in enumerate(CLASS_ORDER):
    print(f"{i:2d} -> {cls}")

print("\nTarget distribution:")
target_counts = df["interest_category"].value_counts().reindex(CLASS_ORDER)
display(target_counts.to_frame("count").assign(percent=lambda x: (x["count"] /
    len(df) * 100).round(2)))
```

Final task classes:

```
0 -> Cell Phones & Smartphones
1 -> Digital Cameras
2 -> Video Games
3 -> Headphones
4 -> Cordless Drills
5 -> Video Game Consoles
6 -> Monitors
7 -> Camcorders
8 -> Mice, Trackballs & Touchpads
9 -> Keyboards & Keypads
```

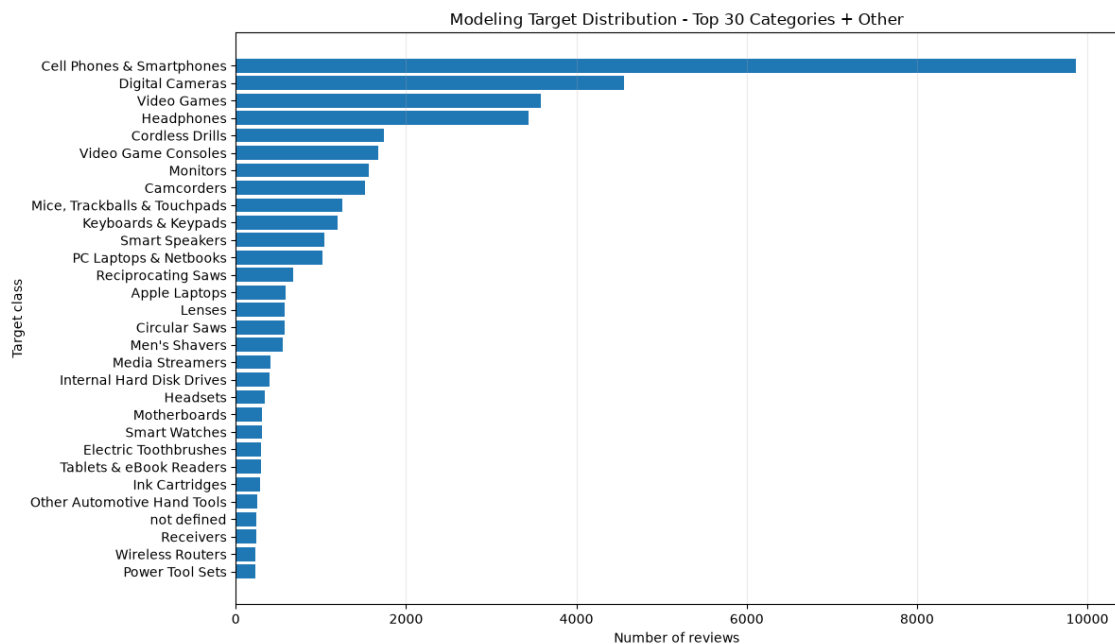
10 -> Smart Speakers  
 11 -> PC Laptops & Netbooks  
 12 -> Reciprocating Saws  
 13 -> Apple Laptops  
 14 -> Lenses  
 15 -> Circular Saws  
 16 -> Men's Shavers  
 17 -> Media Streamers  
 18 -> Internal Hard Disk Drives  
 19 -> Headsets  
 20 -> Motherboards  
 21 -> Smart Watches  
 22 -> Electric Toothbrushes  
 23 -> Tablets & eBook Readers  
 24 -> Ink Cartridges  
 25 -> Other Automotive Hand Tools  
 26 -> not defined  
 27 -> Receivers  
 28 -> Wireless Routers  
 29 -> Power Tool Sets

Target distribution:

| interest_category            | count | percent |
|------------------------------|-------|---------|
| Cell Phones & Smartphones    | 9861  | 25.12   |
| Digital Cameras              | 4565  | 11.63   |
| Video Games                  | 3588  | 9.14    |
| Headphones                   | 3442  | 8.77    |
| Cordless Drills              | 1736  | 4.42    |
| Video Game Consoles          | 1672  | 4.26    |
| Monitors                     | 1563  | 3.98    |
| Camcorders                   | 1514  | 3.86    |
| Mice, Trackballs & Touchpads | 1247  | 3.18    |
| Keyboards & Keypads          | 1201  | 3.06    |
| Smart Speakers               | 1042  | 2.65    |
| PC Laptops & Netbooks        | 1018  | 2.59    |
| Reciprocating Saws           | 678   | 1.73    |
| Apple Laptops                | 583   | 1.49    |
| Lenses                       | 573   | 1.46    |
| Circular Saws                | 573   | 1.46    |
| Men's Shavers                | 557   | 1.42    |
| Media Streamers              | 413   | 1.05    |
| Internal Hard Disk Drives    | 400   | 1.02    |
| Headsets                     | 336   | 0.86    |
| Motherboards                 | 308   | 0.78    |
| Smart Watches                | 306   | 0.78    |
| Electric Toothbrushes        | 297   | 0.76    |

|                             |     |      |
|-----------------------------|-----|------|
| Tablets & eBook Readers     | 295 | 0.75 |
| Ink Cartridges              | 284 | 0.72 |
| Other Automotive Hand Tools | 251 | 0.64 |
| not defined                 | 246 | 0.63 |
| Receivers                   | 243 | 0.62 |
| Wireless Routers            | 232 | 0.59 |
| Power Tool Sets             | 231 | 0.59 |

```
[17]: plt.figure(figsize=(12, 7))
target_plot = target_counts.sort_values()
plt.barh(target_plot.index, target_plot.values)
plt.title("Modeling Target Distribution - Top 30 Categories + Other")
plt.xlabel("Number of reviews")
plt.ylabel("Target class")
plt.grid(axis="x", alpha=0.25)
plt.tight_layout()
plt.savefig(FIG_DIR / "06_modeling_target_distribution.png", dpi=180)
plt.show()
```



## 7 8. Train / Validation / Test Split

We use:

- 70% training
- 15% validation
- 15% test

The split is stratified on the 30-class target so that each class remains represented.

The test set is kept untouched until the final comparison of all four models.

```
[18]: train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    random_state=SEED,
    stratify=df["target_id"],
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=SEED,
    stratify=temp_df["target_id"],
)

print("Train:", train_df.shape)
print("Validation:", val_df.shape)
print("Test:", test_df.shape)

for name, subset in [("Train", train_df), ("Validation", val_df), ("Test",
    test_df)]:
    print(f"\n{name} target distribution:")
    dist = subset["interest_category"].value_counts().reindex(CLASS_ORDER).
    fillna(0)
    print(dist.to_dict())
```

Train: (27478, 12)

Validation: (5888, 12)

Test: (5889, 12)

Train target distribution:

```
{'Cell Phones & Smartphones': 6903, 'Digital Cameras': 3195, 'Video Games':
2512, 'Headphones': 2409, 'Cordless Drills': 1215, 'Video Game Consoles': 1170,
'Monitors': 1094, 'Camcorders': 1060, 'Mice, Trackballs & Touchpads': 873,
'Keyboards & Keypads': 841, 'Smart Speakers': 729, 'PC Laptops & Netbooks': 713,
'Reciprocating Saws': 475, 'Apple Laptops': 408, 'Lenses': 401, 'Circular Saws':
401, 'Men's Shavers': 390, 'Media Streamers': 289, 'Internal Hard Disk Drives':
280, 'Headsets': 235, 'Motherboards': 216, 'Smart Watches': 214, 'Electric
Toothbrushes': 208, 'Tablets & eBook Readers': 206, 'Ink Cartridges': 199,
'Other Automotive Hand Tools': 176, 'not defined': 172, 'Receivers': 170,
'Wireless Routers': 162, 'Power Tool Sets': 162}
```

Validation target distribution:

```
{'Cell Phones & Smartphones': 1479, 'Digital Cameras': 685, 'Video Games': 538,
'Headphones': 516, 'Cordless Drills': 260, 'Video Game Consoles': 251,
```

```
'Monitors': 234, 'Camcorders': 227, 'Mice, Trackballs & Touchpads': 187,
'Keyboards & Keypads': 180, 'Smart Speakers': 156, 'PC Laptops & Netbooks': 152,
'Reciprocating Saws': 101, 'Apple Laptops': 87, 'Lenses': 86, 'Circular Saws':
86, 'Men's Shavers': 83, 'Media Streamers': 62, 'Internal Hard Disk Drives': 60,
'Headsets': 51, 'Motherboards': 46, 'Smart Watches': 46, 'Electric
Toothbrushes': 45, 'Tablets & eBook Readers': 45, 'Ink Cartridges': 43, 'Other
Automotive Hand Tools': 38, 'not defined': 37, 'Receivers': 37, 'Wireless
Routers': 35, 'Power Tool Sets': 35}
```

Test target distribution:

```
{'Cell Phones & Smartphones': 1479, 'Digital Cameras': 685, 'Video Games': 538,
'Headphones': 517, 'Cordless Drills': 261, 'Video Game Consoles': 251,
'Monitors': 235, 'Camcorders': 227, 'Mice, Trackballs & Touchpads': 187,
'Keyboards & Keypads': 180, 'Smart Speakers': 157, 'PC Laptops & Netbooks': 153,
'Reciprocating Saws': 102, 'Apple Laptops': 88, 'Lenses': 86, 'Circular Saws':
86, 'Men's Shavers': 84, 'Media Streamers': 62, 'Internal Hard Disk Drives': 60,
'Headsets': 50, 'Motherboards': 46, 'Smart Watches': 46, 'Electric
Toothbrushes': 44, 'Tablets & eBook Readers': 44, 'Ink Cartridges': 42, 'Other
Automotive Hand Tools': 37, 'not defined': 37, 'Receivers': 36, 'Wireless
Routers': 35, 'Power Tool Sets': 34}
```

## 8 9. Leakage-Safe Preprocessing and Text Feature Selection

All feature-learning transformations are fitted on the **training set only**.

This is essential because fitting a vectorizer, scaler, vocabulary, or feature selector using validation/test data would allow information from unseen data to influence the representation.

### 8.0.1 Traditional ML representation

1. Fit TF-IDF on `X_train`
2. Transform validation and test with the same vectorizer
3. Fit chi-square `SelectKBest` on training data only
4. Transform validation/test with the same selector
5. Fit a rating scaler on training ratings only
6. Append the scaled rating to the selected TF-IDF matrix

The resulting `X` matrices are sparse numerical representations.

```
[19]: X_train_text = train_df["review_text"].tolist()
X_val_text = val_df["review_text"].tolist()
X_test_text = test_df["review_text"].tolist()

y_train = train_df["target_id"].to_numpy(dtype=np.int64)
y_val = val_df["target_id"].to_numpy(dtype=np.int64)
y_test = test_df["target_id"].to_numpy(dtype=np.int64)

print("Training examples:", len(X_train_text))
print("Validation examples:", len(X_val_text))
```

```
print("Testing examples:", len(X_test_text))
```

Training examples: 27478  
Validation examples: 5888  
Testing examples: 5889

```
[20]: TFIDF_MAX_FEATURES = 30000
TFIDF_MIN_DF = 2
TFIDF_MAX_DF = 0.98

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=TFIDF_MIN_DF,
    max_df=TFIDF_MAX_DF,
    max_features=TFIDF_MAX_FEATURES,
    sublinear_tf=True,
)

X_train_tfidf = vectorizer.fit_transform(X_train_text)
X_val_tfidf = vectorizer.transform(X_val_text)
X_test_tfidf = vectorizer.transform(X_test_text)

print("TF-IDF feature count before selection:", X_train_tfidf.shape[1])
print("Train matrix shape:", X_train_tfidf.shape)
print("Validation matrix shape:", X_val_tfidf.shape)
print("Test matrix shape:", X_test_tfidf.shape)
```

TF-IDF feature count before selection: 30000  
Train matrix shape: (27478, 30000)  
Validation matrix shape: (5888, 30000)  
Test matrix shape: (5889, 30000)

```
[21]: SELECTED_TFIDF_FEATURES = min(12000, X_train_tfidf.shape[1])

selector = SelectKBest(
    score_func=chi2,
    k=SELECTED_TFIDF_FEATURES,
)

X_train_text_sel = selector.fit_transform(X_train_tfidf, y_train)
X_val_text_sel = selector.transform(X_val_tfidf)
X_test_text_sel = selector.transform(X_test_tfidf)

selected_feature_names = np.asarray(vectorizer.
    ↪get_feature_names_out())[selector.get_support()]
```

```

print("Selected TF-IDF features:", len(selected_feature_names))
print("Selected matrix shape:", X_train_text_sel.shape)

print("\nExamples of selected high-information terms:")
print(selected_feature_names[:80])

```

Selected TF-IDF features: 12000  
Selected matrix shape: (27478, 12000)

Examples of selected high-information terms:

```

['00 good' '000' '01' '04' '0ah' '10' '10 10' '10 days' '10 hours'
 '10 key' '10 laptop' '10 pro' '10 sep' '10 years' '100 00' '100 hard'
 '100 recommended' '100 time' '100 worth' '100mm' '105' '105 lens' '105mm'
 '1080' '1080i' '1080p' '1080p 60fps' '10d' '10th' '10x' '110' '12'
 '12 megapixel' '12 mp' '12 volt' '120 gb' '1200' '120hz' '125' '128'
 '128 gb' '128gb' '12gb' '12v' '13' '13 inch' '13 macbook' '135' '135mm'
 '14' '14 years' '144' '1440p' '144hz' '15' '15 inch' '15 macbook'
 '15 sep' '15 year' '15 years' '16' '16 10' '16 35' '16 50' '16 years'
 '1600' '1600 1200' '16gb' '17' '18' '18 105' '18 105mm' '18 135mm'
 '18 200' '18 200mm' '18 55' '18 55mm' '18 volt' '18mm' '18v']

```

```

[22]: # Rating is an auxiliary numeric feature. The scaler is fitted only on training
      ↪ data.
rating_scaler = StandardScaler()

rating_train = rating_scaler.fit_transform(
    train_df[["rating"]].to_numpy(dtype=np.float64)
)
rating_val = rating_scaler.transform(
    val_df[["rating"]].to_numpy(dtype=np.float64)
)
rating_test = rating_scaler.transform(
    test_df[["rating"]].to_numpy(dtype=np.float64)
)

# MultinomialNB needs non-negative features. To keep one common representation,
# use a [0, 1] min-max style mapping for the auxiliary rating instead of a
↪ negative z-score.
rating_minmax_train = (train_df["rating"].to_numpy(dtype=np.float64) - 1.0) / 4.
↪ 0
rating_minmax_val = (val_df["rating"].to_numpy(dtype=np.float64) - 1.0) / 4.0
rating_minmax_test = (test_df["rating"].to_numpy(dtype=np.float64) - 1.0) / 4.0

X_train_ml = hstack(
    [X_train_text_sel, csr_matrix(rating_minmax_train.reshape(-1, 1))],
    format="csr",
)

```

```

X_val_ml = hstack(
    [X_val_text_sel, csr_matrix(rating_minmax_val.reshape(-1, 1))],
    format="csr",
)
X_test_ml = hstack(
    [X_test_text_sel, csr_matrix(rating_minmax_test.reshape(-1, 1))],
    format="csr",
)

print("Final ML input matrix:")
print("X_train:", X_train_ml.shape)
print("X_val  :", X_val_ml.shape)
print("X_test :", X_test_ml.shape)
print("Non-zero values in X_train:", X_train_ml.nnz)

```

```

Final ML input matrix:
X_train: (27478, 12001)
X_val  : (5888, 12001)
X_test : (5889, 12001)
Non-zero values in X_train: 517340

```

### 8.0.2 Feature Selection Check

For text, “feature engineering” is not limited to inventing new columns.

It also means choosing **how language becomes features** and removing dimensions that are unlikely to contribute enough signal.

The pipeline therefore performs:

```

word / bigram candidate features
    ↓
TF-IDF weighting
    ↓
Chi-square relevance test
    ↓
20,000 selected text features
    +
1 compact rating feature

```

This gives the ML models a controlled representation instead of passing the entire raw vocabulary to the classifier.

```

[23]: feature_score_df = pd.DataFrame({
        "feature": vectorizer.get_feature_names_out(),
        "chi2_score": selector.scores_,
    }).dropna().sort_values("chi2_score", ascending=False)

display(feature_score_df.head(30))

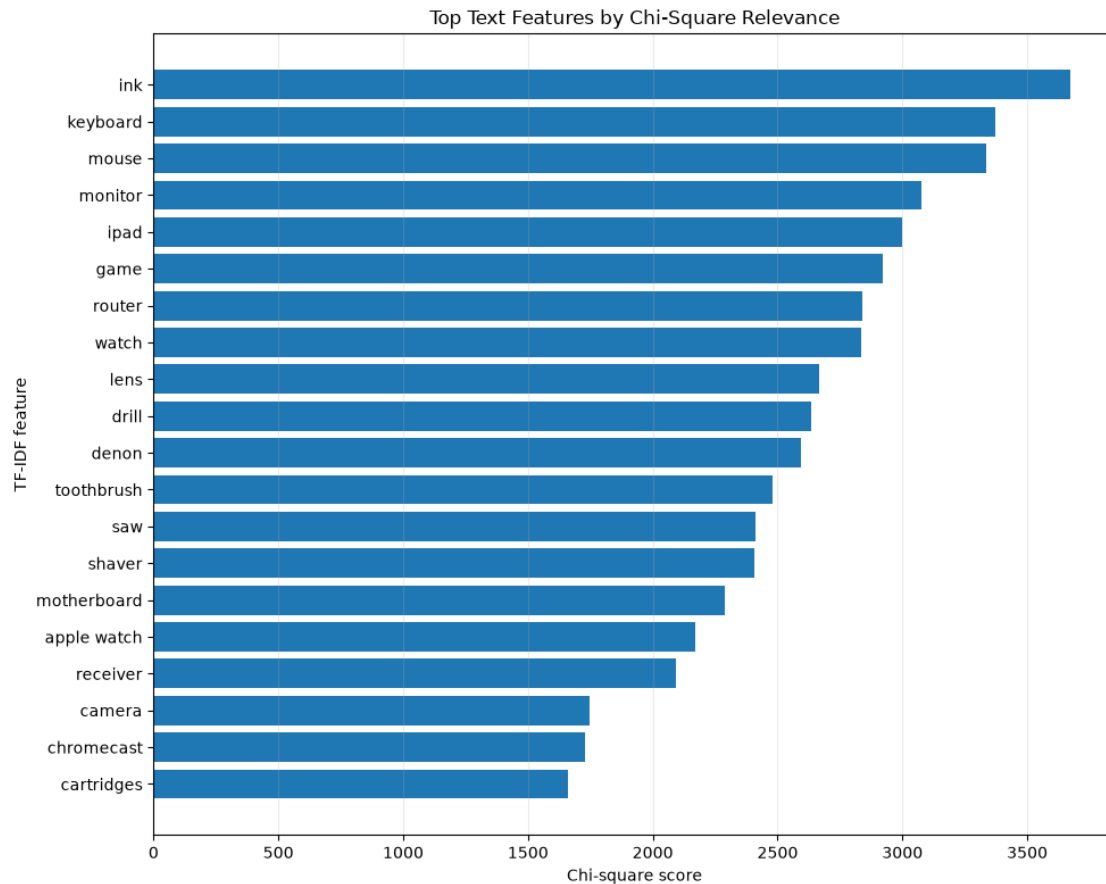
```

```

plt.figure(figsize=(10, 8))
plot_df = feature_score_df.head(20).sort_values("chi2_score")
plt.barh(plot_df["feature"], plot_df["chi2_score"])
plt.title("Top Text Features by Chi-Square Relevance")
plt.xlabel("Chi-square score")
plt.ylabel("TF-IDF feature")
plt.grid(axis="x", alpha=0.25)
plt.tight_layout()
plt.savefig(FIG_DIR / "07_top_text_features_chi2.png", dpi=180)
plt.show()

```

|       | feature     | chi2_score  |
|-------|-------------|-------------|
| 13404 | ink         | 3671.098116 |
| 14232 | keyboard    | 3373.785315 |
| 16811 | mouse       | 3333.377385 |
| 16705 | monitor     | 3075.479014 |
| 13613 | ipad        | 2996.796477 |
| 9839  | game        | 2922.173141 |
| 22845 | router      | 2838.240245 |
| 29083 | watch       | 2836.417238 |
| 14815 | lens        | 2667.032482 |
| 6860  | drill       | 2633.833170 |
| 5826  | denon       | 2593.526284 |
| 27639 | toothbrush  | 2479.829769 |
| 23207 | saw         | 2409.876768 |
| 24858 | shaver      | 2405.591616 |
| 16788 | motherboard | 2288.498313 |
| 1293  | apple watch | 2171.894722 |
| 21986 | receiver    | 2092.946249 |
| 3132  | camera      | 1744.581448 |
| 4014  | chromecast  | 1728.652680 |
| 3720  | cartridges  | 1661.486043 |
| 24849 | shave       | 1640.199058 |
| 20400 | printer     | 1625.727181 |
| 16047 | macbook     | 1623.515984 |
| 6930  | drive       | 1602.081170 |
| 18633 | phone       | 1586.970876 |
| 14863 | lens sep    | 1575.131296 |
| 11106 | gopro       | 1565.733610 |
| 16851 | mouse sep   | 1500.688374 |
| 17546 | norelco     | 1487.549521 |
| 25866 | sonicare    | 1409.684187 |



### 8.0.3 Representation Example - One Review

A2 requires the representation to be visible, not hidden inside model code.

The next cell shows:

Original review

↓

Normalized text

↓

Selected TF-IDF vector shape

For the Deep Learning path, a later cell will show:

Tokens → Token IDs → Embedding tensor → pooled vector

```
[24]: example_idx = 0
print("Original title:")
print(train_df.iloc[example_idx]["review title"])
print("\nOriginal content:")
print(train_df.iloc[example_idx]["review content"])
```

```

print("\nNormalized review:")
print(train_df.iloc[example_idx]["review_text"])

print("\nTarget:")
print(train_df.iloc[example_idx]["interest_category"])

print("\nTraditional ML vector shape for one review:")
print(X_train_ml[example_idx].shape)

print("\nNumber of non-zero values in the vector:")
print(X_train_ml[example_idx].nnz)

```

Original title:

Great phone

Original content:

Phone was in great used condition. I haven't taken photos yet or used all the features, but it seems to be working just fine. I do find I have to charge the battery daily, though.

Normalized review:

great phone [SEP] phone was in great used condition. i haven't taken photos yet or used all the features, but it seems to be working just fine. i do find i have to charge the battery daily, though.

Target:

Cell Phones & Smartphones

Traditional ML vector shape for one review:

(1, 12001)

Number of non-zero values in the vector:

17

## 9 10. Model 1 - Logistic Regression

Logistic Regression is a strong linear baseline for high-dimensional TF-IDF text features.

It is useful here because: - it works well with sparse text vectors; - its coefficients can be inspected as feature evidence; - it can output class probabilities for deployment.

The model uses the same leakage-safe `X_train_ml` representation as the other ML models.

```

[25]: ml_models = {}
      ml_predictions = {}
      ml_probabilities = {}
      ml_training_times = {}

```

```

ml_results = []

start = time.time()

logreg = OneVsRestClassifier(
    LogisticRegression(
        C=4.0,
        max_iter=300,
        solver="liblinear",
        random_state=SEED,
    ),
    n_jobs=1,
)

logreg.fit(X_train_ml, y_train)
logreg_time = time.time() - start

logreg_pred = logreg.predict(X_test_ml)
logreg_prob = logreg.predict_proba(X_test_ml)

ml_models["Logistic Regression"] = logreg
ml_predictions["Logistic Regression"] = logreg_pred
ml_probabilities["Logistic Regression"] = logreg_prob
ml_training_times["Logistic Regression"] = logreg_time

print("Training time:", round(logreg_time, 2), "seconds")
print("Prediction shape:", logreg_pred.shape)
print("Probability shape:", logreg_prob.shape)

```

```

Training time: 7.49 seconds
Prediction shape: (5889,)
Probability shape: (5889, 30)

```

## 10 11. Model 2 - Linear SVM

Linear SVM is another strong text-classification baseline.

It usually handles sparse TF-IDF representations efficiently and directly optimizes a large-margin decision boundary.

```

[26]: start = time.time()

svm_model = LinearSVC(
    C=1.5,
    random_state=SEED,
)

svm_model.fit(X_train_ml, y_train)

```

```

svm_time = time.time() - start

svm_pred = svm_model.predict(X_test_ml)
svm_scores = svm_model.decision_function(X_test_ml)

ml_models["Linear SVM"] = svm_model
ml_predictions["Linear SVM"] = svm_pred
ml_training_times["Linear SVM"] = svm_time

print("Training time:", round(svm_time, 2), "seconds")
print("Decision score shape:", svm_scores.shape)

```

Training time: 10.74 seconds  
Decision score shape: (5889, 30)

## 11 12. Model 3 - Multinomial Naive Bayes

Multinomial Naive Bayes is a classic text-classification baseline.

It is computationally light and works naturally with non-negative TF-IDF-style features. The shared ML representation therefore keeps all inputs non-negative.

```

[27]: start = time.time()

nb_model = MultinomialNB(alpha=0.25)

nb_model.fit(X_train_ml, y_train)
nb_time = time.time() - start

nb_pred = nb_model.predict(X_test_ml)
nb_prob = nb_model.predict_proba(X_test_ml)

ml_models["Multinomial Naive Bayes"] = nb_model
ml_predictions["Multinomial Naive Bayes"] = nb_pred
ml_probabilities["Multinomial Naive Bayes"] = nb_prob
ml_training_times["Multinomial Naive Bayes"] = nb_time

print("Training time:", round(nb_time, 2), "seconds")
print("Probability shape:", nb_prob.shape)

```

Training time: 0.05 seconds  
Probability shape: (5889, 30)

## 12 13. Evaluate the Three Traditional ML Models

All three traditional models are evaluated on the same untouched test set.

For the multi-class task, the main comparison metrics are:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1
- Balanced Accuracy
- One-vs-Rest ROC-AUC where a probability/score representation is available

Macro metrics are emphasized because the target is not perfectly balanced.

```
[28]: def softmax_numpy(scores):
    scores = np.asarray(scores, dtype=np.float64)
    shifted = scores - np.max(scores, axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    return exp_scores / np.sum(exp_scores, axis=1, keepdims=True)

# Convert Linear SVM decision scores to normalized class scores for ROC-AUC /
↳display.
# These are NOT calibrated probabilities.
svm_pseudo_prob = softmax_numpy(svm_scores)

ml_probabilities["Linear SVM (normalized scores)"] = svm_pseudo_prob

def classification_metrics(y_true, y_pred, prob=None):
    result = {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision_macro": precision_score(y_true, y_pred, average="macro",
↳zero_division=0),
        "Recall_macro": recall_score(y_true, y_pred, average="macro",
↳zero_division=0),
        "F1_macro": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "Balanced_Accuracy": balanced_accuracy_score(y_true, y_pred),
    }
    if prob is not None:
        try:
            y_bin = label_binarize(y_true, classes=np.arange(len(CLASS_ORDER)))
            result["ROC_AUC_OVR_macro"] = roc_auc_score(
                y_bin,
                prob,
                multi_class="ovr",
                average="macro",
            )
        except ValueError:
            result["ROC_AUC_OVR_macro"] = np.nan
    else:
        result["ROC_AUC_OVR_macro"] = np.nan
    return result

ml_results = []
```

```

for name in ["Logistic Regression", "Linear SVM", "Multinomial Naive Bayes"]:
    pred = ml_predictions[name]
    prob = (
        ml_probabilities[name]
        if name in ml_probabilities
        else ml_probabilities["Linear SVM (normalized scores)"]
    )

    metrics = classification_metrics(y_test, pred, prob)
    metrics["Model"] = name
    metrics["Training_Time_sec"] = ml_training_times[name]
    ml_results.append(metrics)

traditional_results = pd.DataFrame(ml_results)[
    [
        "Model",
        "Accuracy",
        "Precision_macro",
        "Recall_macro",
        "F1_macro",
        "Balanced_Accuracy",
        "ROC_AUC_OVR_macro",
        "Training_Time_sec",
    ]
].sort_values("F1_macro", ascending=False).reset_index(drop=True)

display(traditional_results)

```

|   | Model                   | Accuracy | Precision_macro | Recall_macro | F1_macro | \ |
|---|-------------------------|----------|-----------------|--------------|----------|---|
| 0 | Linear SVM              | 0.776193 | 0.781943        | 0.629196     | 0.690231 |   |
| 1 | Logistic Regression     | 0.771268 | 0.854291        | 0.596947     | 0.686306 |   |
| 2 | Multinomial Naive Bayes | 0.688232 | 0.888586        | 0.391919     | 0.483426 |   |

|   | Balanced_Accuracy | ROC_AUC_OVR_macro | Training_Time_sec |
|---|-------------------|-------------------|-------------------|
| 0 | 0.629196          | 0.944697          | 10.740268         |
| 1 | 0.596947          | 0.957819          | 7.488329          |
| 2 | 0.391919          | 0.914931          | 0.050471          |

```

[29]: plot_metrics = [
        "Accuracy",
        "Precision_macro",
        "Recall_macro",
        "Balanced_Accuracy",
    ]

fig, axes = plt.subplots(2, 2, figsize=(15, 10))

```

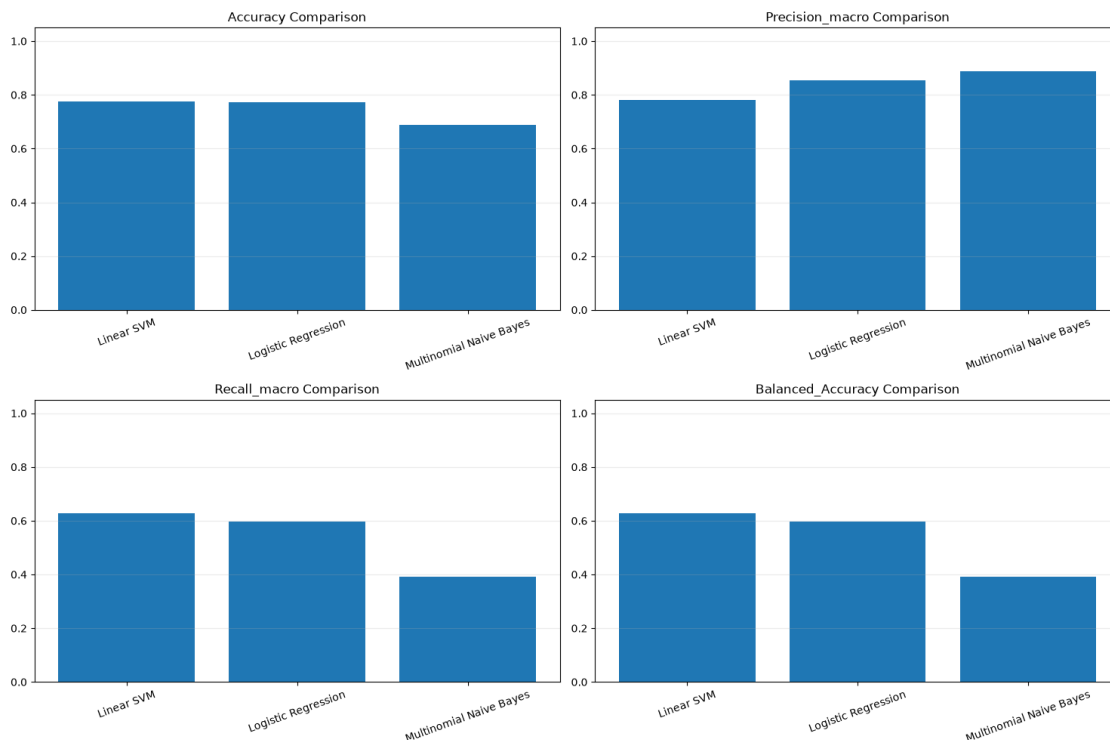
```

axes = axes.ravel()

for ax, metric in zip(axes, plot_metrics):
    axes_idx = list(axes).index(ax)
    vals = traditional_results[metric].to_numpy()
    ax.bar(traditional_results["Model"], vals)
    ax.set_title(f"{metric} Comparison")
    ax.set_ylim(0, 1.05)
    ax.tick_params(axis="x", rotation=20)
    ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
plt.savefig(FIG_DIR / "08_traditional_ml_metric_comparison.png", dpi=180)
plt.show()

```



### 12.0.1 Best ML Validation Rule

The notebook selects the strongest traditional ML model using **Macro F1** because: - the target contains a majority class and an **Other** class; - macro averaging gives every class equal metric weight; - a model that predicts the majority classes well but ignores smaller classes should not automatically win.

The final four-model comparison will still show all metrics, so selection is transparent.

```
[30]: best_ml_name = traditional_results.iloc[0]["Model"]
print("Best traditional ML model by Macro F1:", best_ml_name)
```

Best traditional ML model by Macro F1: Linear SVM

## 12.0.2 Detailed Test Evaluation - Best Traditional ML

A confusion matrix shows where category interests are confused with each other.

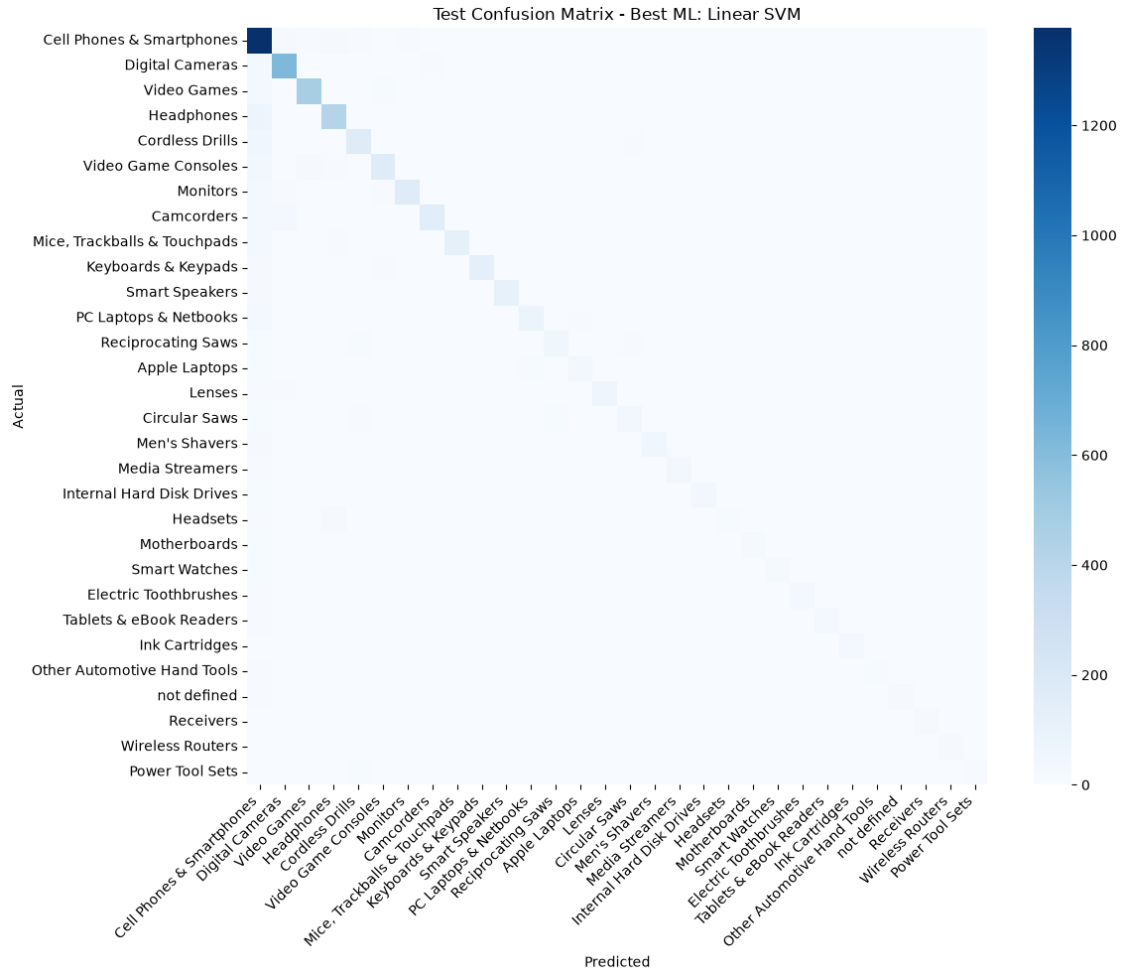
This is especially useful for text classification because related product categories often use similar vocabulary.

```
[31]: best_ml_pred = ml_predictions[best_ml_name]

if best_ml_name == "Linear SVM":
    best_ml_prob = ml_probabilities["Linear SVM (normalized scores)"]
elif best_ml_name in ml_probabilities:
    best_ml_prob = ml_probabilities[best_ml_name]
else:
    best_ml_prob = None

cm_ml = confusion_matrix(y_test, best_ml_pred, labels=np.
    ↳arange(len(CLASS_ORDER)))

plt.figure(figsize=(12, 10))
sns.heatmap(
    cm_ml,
    annot=False,
    cmap="Blues",
    xticklabels=CLASS_ORDER,
    yticklabels=CLASS_ORDER,
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Test Confusion Matrix - Best ML: {best_ml_name}")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig(FIG_DIR / "09_best_ml_confusion_matrix.png", dpi=180)
plt.show()
```



### 12.0.3 Best ML Classification Report

This report makes per-class behavior visible instead of relying only on aggregate metrics.

```
[32]: print(
    classification_report(
        y_test,
        best_ml_pred,
        labels=np.arange(len(CLASS_ORDER)),
        target_names=CLASS_ORDER,
        zero_division=0,
    )
)
```

|                           | precision | recall | f1-score | support |
|---------------------------|-----------|--------|----------|---------|
| Cell Phones & Smartphones | 0.70      | 0.93   | 0.80     | 1479    |
| Digital Cameras           | 0.87      | 0.91   | 0.89     | 685     |

|                              |      |      |      |      |
|------------------------------|------|------|------|------|
| Video Games                  | 0.90 | 0.88 | 0.89 | 538  |
| Headphones                   | 0.80 | 0.81 | 0.81 | 517  |
| Cordless Drills              | 0.71 | 0.65 | 0.68 | 261  |
| Video Game Consoles          | 0.74 | 0.65 | 0.69 | 251  |
| Monitors                     | 0.78 | 0.72 | 0.75 | 235  |
| Camcorders                   | 0.87 | 0.67 | 0.75 | 227  |
| Mice, Trackballs & Touchpads | 0.86 | 0.65 | 0.74 | 187  |
| Keyboards & Keypads          | 0.85 | 0.72 | 0.78 | 180  |
| Smart Speakers               | 0.83 | 0.71 | 0.77 | 157  |
| PC Laptops & Netbooks        | 0.61 | 0.54 | 0.58 | 153  |
| Reciprocating Saws           | 0.64 | 0.55 | 0.59 | 102  |
| Apple Laptops                | 0.71 | 0.51 | 0.60 | 88   |
| Lenses                       | 0.90 | 0.76 | 0.82 | 86   |
| Circular Saws                | 0.67 | 0.53 | 0.59 | 86   |
| Men's Shavers                | 0.80 | 0.63 | 0.71 | 84   |
| Media Streamers              | 0.84 | 0.66 | 0.74 | 62   |
| Internal Hard Disk Drives    | 0.73 | 0.63 | 0.68 | 60   |
| Headsets                     | 0.64 | 0.28 | 0.39 | 50   |
| Motherboards                 | 0.77 | 0.50 | 0.61 | 46   |
| Smart Watches                | 0.76 | 0.57 | 0.65 | 46   |
| Electric Toothbrushes        | 0.97 | 0.66 | 0.78 | 44   |
| Tablets & eBook Readers      | 0.82 | 0.61 | 0.70 | 44   |
| Ink Cartridges               | 0.97 | 0.86 | 0.91 | 42   |
| Other Automotive Hand Tools  | 0.57 | 0.32 | 0.41 | 37   |
| not defined                  | 0.84 | 0.57 | 0.68 | 37   |
| Receivers                    | 0.83 | 0.56 | 0.67 | 36   |
| Wireless Routers             | 0.91 | 0.57 | 0.70 | 35   |
| Power Tool Sets              | 0.56 | 0.26 | 0.36 | 34   |
| accuracy                     |      |      | 0.78 | 5889 |
| macro avg                    | 0.78 | 0.63 | 0.69 | 5889 |
| weighted avg                 | 0.78 | 0.78 | 0.77 | 5889 |

```
[33]: # Top text features for Logistic Regression, if the model exists.
if "Logistic Regression" in ml_models:
    lr_estimators = ml_models["Logistic Regression"].estimators_
    lr_coef = np.vstack([est.coef_.ravel() for est in lr_estimators])
    mean_abs_coef = np.mean(np.abs(lr_coef), axis=0)

    text_feature_names_final = list(selected_feature_names) +
↳ ["rating_auxiliary"]
    coef_df = pd.DataFrame({
        "feature": text_feature_names_final,
        "mean_abs_coefficient": mean_abs_coef,
    }).sort_values("mean_abs_coefficient", ascending=False)
```

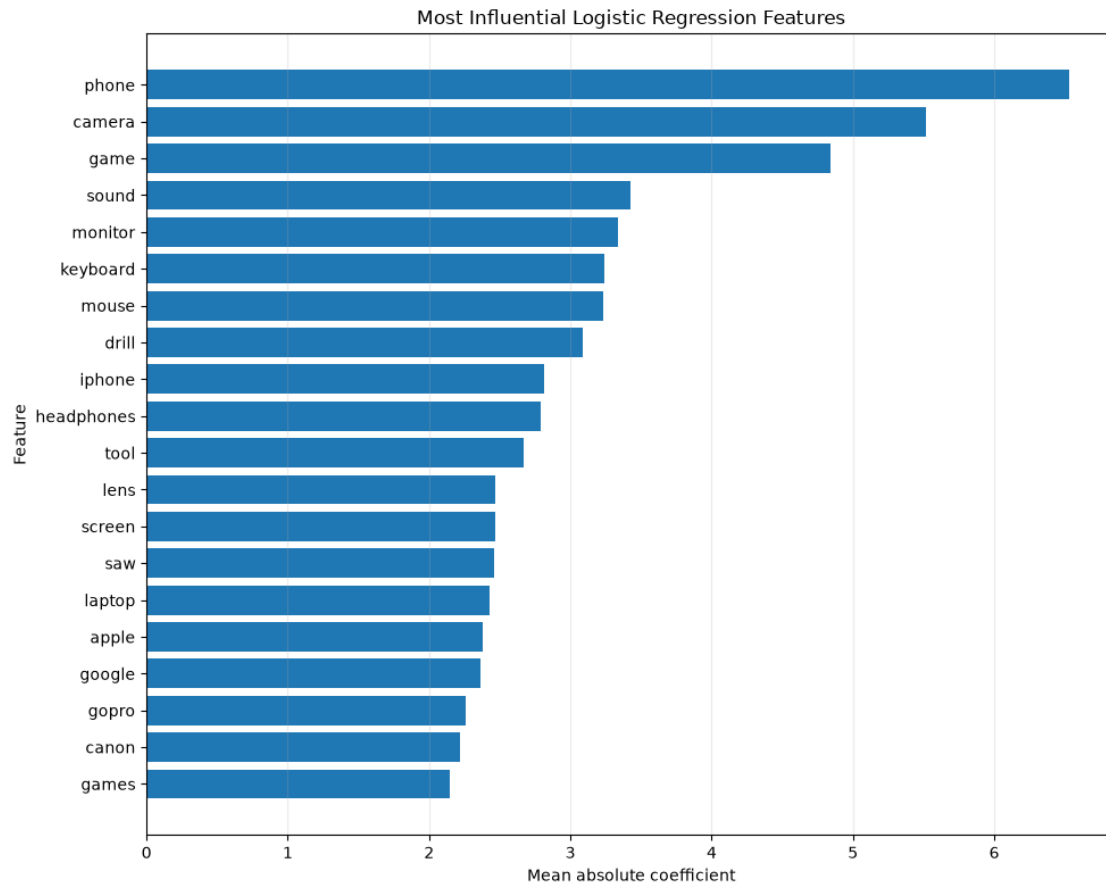
```

display(coef_df.head(25))

plt.figure(figsize=(10, 8))
top_coef = coef_df.head(20).sort_values("mean_abs_coefficient")
plt.barh(top_coef["feature"], top_coef["mean_abs_coefficient"])
plt.title("Most Influential Logistic Regression Features")
plt.xlabel("Mean absolute coefficient")
plt.ylabel("Feature")
plt.grid(axis="x", alpha=0.25)
plt.tight_layout()
plt.savefig(FIG_DIR / "10_logistic_top_features.png", dpi=180)
plt.show()

```

|       | feature    | mean_abs_coefficient |
|-------|------------|----------------------|
| 7521  | phone      | 6.533285             |
| 1348  | camera     | 5.516461             |
| 3808  | game       | 4.844364             |
| 10281 | sound      | 3.423527             |
| 6680  | monitor    | 3.334904             |
| 5602  | keyboard   | 3.244222             |
| 6766  | mouse      | 3.235966             |
| 2687  | drill      | 3.084635             |
| 5419  | iphone     | 2.814058             |
| 4929  | headphones | 2.787696             |
| 10926 | tool       | 2.668115             |
| 5902  | lens       | 2.471185             |
| 9220  | screen     | 2.466969             |
| 9132  | saw        | 2.459827             |
| 5784  | laptop     | 2.429594             |
| 545   | apple      | 2.376857             |
| 4299  | google     | 2.365284             |
| 4342  | gopro      | 2.261923             |
| 1493  | canon      | 2.216266             |
| 3943  | games      | 2.144447             |
| 798   | battery    | 2.131982             |
| 10241 | sony       | 1.904023             |
| 10951 | tools      | 1.834767             |
| 11554 | watch      | 1.733924             |
| 1918  | computer   | 1.722797             |



## 13 14. Deep Learning from Scratch - Concept

The lecture material defines Deep Learning as learning a composition of functions:

$X \rightarrow f_1 \rightarrow H_1 \rightarrow f_2 \rightarrow H_2 \rightarrow f_3 \rightarrow \text{prediction}$

For the supplied tutorial, the dense network follows:

$$Z_1 = XW_1 + b_1$$

$$H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2$$

$$H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3$$

$$\hat{y} = \text{Softmax}(Z_3)$$

For the Comments application, the input is text, so we add an explicit learnable embedding representation before the dense layers:

Token IDs

$\downarrow$   
 Embedding matrix  
 $\downarrow$   
 Mean pooled representation  
 $\downarrow$   
 Dense + ReLU  
 $\downarrow$   
 Dense + ReLU  
 $\downarrow$   
 Softmax

This keeps the fundamental lecture pattern while adapting the input representation to text data.

## 14 15. Deep Learning Mathematics

For one mini-batch of token IDs:

```

E = Embedding(TokenIDs)
H0 = MeanPool(E)

```

Then the dense part is:

```

Z1 = H0 W1 + b1
H1 = ReLU(Z1)

```

```

Z2 = H1 W2 + b2
H2 = ReLU(Z2)

```

```

Z3 = H2 W3 + b3
P = Softmax(Z3)

```

### 14.0.1 Multi-class Cross-Entropy

For true class  $y_i$  and predicted class probabilities  $P_i$ :

$$L = -(1/N) \sum \log(P_i, y_i)$$

### 14.0.2 Learning

The parameters are:

```

= {Embedding, W1, b1, W2, b2, W3, b3}

```

and gradient descent applies:

```

← - L

```

For Softmax + cross-entropy, the output gradient is:

$$dZ3 = (P - Y) / N$$

The rest follows the chain rule backward through the ReLU layers and mean-pooling embedding representation.

The important conceptual flow remains:

Forward → Loss → Backpropagation → Gradient Descent → Repeat

## 15 16. Prepare the Data for the NumPy Neural Network

We do **not** reuse the TF-IDF matrix as the neural-network input because the A2 text representation requirement should be made explicit.

Instead, the neural network builds a vocabulary from the training reviews only:

```
training text
  ↓
token frequency counting
  ↓
vocabulary
  ↓
integer token IDs
  ↓
fixed-length sequences
```

Special tokens:

0 = PAD

1 = UNK

A review longer than the maximum sequence length is truncated for the neural-network representation. This is a representation constraint, not deletion from the original CSV.

```
[34]: TOKEN_PATTERN = re.compile(r"[a-z0-9']+")
MAX_VOCAB_SIZE = 4000
MIN_TOKEN_FREQ = 2
MAX_LEN = 40

def tokenize(text):
    return TOKEN_PATTERN.findall(str(text).lower())

train_tokens = [tokenize(x) for x in X_train_text]

token_counter = Counter()
for toks in train_tokens:
    token_counter.update(toks)

vocab_tokens = [
    tok for tok, freq in token_counter.most_common()
    if freq >= MIN_TOKEN_FREQ
][: MAX_VOCAB_SIZE - 2]

PAD_ID = 0
UNK_ID = 1
```

```

token_to_id = {"<PAD>": PAD_ID, "<UNK>": UNK_ID}
for tok in vocab_tokens:
    token_to_id[tok] = len(token_to_id)

id_to_token = {idx: tok for tok, idx in token_to_id.items()}

print("Vocabulary size:", len(token_to_id))
print("Most common tokens:", token_counter.most_common(20))

```

Vocabulary size: 4000

Most common tokens: [('the', 49259), ('and', 31101), ('i', 29326), ('sep', 27478), ('a', 27186), ('to', 25323), ('it', 25225), ('is', 20088), ('great', 16683), ('for', 15909), ('this', 13930), ('of', 13184), ('with', 11165), ('my', 10538), ('good', 10202), ('in', 9841), ('you', 8831), ('but', 8513), ('that', 8081), ('camera', 7944)]

```

[35]: def encode_text(text, max_len=MAX_LEN):
    tokens = tokenize(text)
    ids = [token_to_id.get(tok, UNK_ID) for tok in tokens[:max_len]]

    if len(ids) < max_len:
        ids = ids + [PAD_ID] * (max_len - len(ids))

    return np.asarray(ids, dtype=np.int64)

X_train_ids = np.stack([encode_text(x) for x in X_train_text])
X_val_ids = np.stack([encode_text(x) for x in X_val_text])
X_test_ids = np.stack([encode_text(x) for x in X_test_text])

print("Token ID representation shapes:")
print("X_train_ids:", X_train_ids.shape)
print("X_val_ids :", X_val_ids.shape)
print("X_test_ids :", X_test_ids.shape)

```

Token ID representation shapes:

X\_train\_ids: (27478, 40)

X\_val\_ids : (5888, 40)

X\_test\_ids : (5889, 40)

### 15.0.1 Text Representation Example

For one review we explicitly inspect:

Raw review

↓

tokens

↓

token IDs

↓  
embedding lookup  
↓  
mean pooled vector

The actual embedding weights are initialized randomly in the next step, then learned by back-propagation. This is why the embedding is a **learned representation**, not a fixed hand-crafted encoding.

```
[36]: demo_text = X_train_text[0]
demo_tokens = tokenize(demo_text)
demo_ids = encode_text(demo_text)

print("Demo text:")
print(demo_text[:1000])

print("\nFirst 25 tokens:")
print(demo_tokens[:25])

print("\nFirst 25 token IDs:")
print(demo_ids[:25])

print("\nSequence shape:", demo_ids.shape)
```

Demo text:

great phone [SEP] phone was in great used condition. i haven't taken photos yet or used all the features, but it seems to be working just fine. i do find i have to charge the battery daily, though.

First 25 tokens:

['great', 'phone', 'sep', 'phone', 'was', 'in', 'great', 'used', 'condition', 'i', 'haven', 't', 'taken', 'photos', 'yet', 'or', 'used', 'all', 'the', 'features', 'but', 'it', 'seems', 'to', 'be']

First 25 token IDs:

```
[ 10  24   5  24  22  17  10  82  70   4 2074 139 662 206
 274  58  82  36   2  97  19   8 284   7  40]
```

Sequence shape: (40,)

## 16 17. Step 1 - Implement Activation Functions

The hidden layers use **ReLU**:

$\text{ReLU}(x) = \max(0, x)$

The output layer uses **Softmax** because the task has multiple interest categories:

$\text{Softmax}(z_j) = \exp(z_j) / \sum \exp(z_k)$

A stable Softmax subtracts the row maximum before exponentiation.

A derivative function for ReLU is required by manual backpropagation.

```
[37]: def relu(x):
        return np.maximum(0.0, x)

def relu_derivative(x):
    return (x > 0).astype(np.float64)

def softmax(x):
    x = np.asarray(x, dtype=np.float64)
    shifted = x - np.max(x, axis=1, keepdims=True)
    exp_x = np.exp(shifted)
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# Small activation-function verification
activation_demo = np.array([[ -2.0, 0.0, 1.5]])
print("ReLU:", relu(activation_demo))
print("ReLU derivative:", relu_derivative(activation_demo))

softmax_demo = softmax(np.array([[1.0, 2.0, 3.0]]))
print("Softmax:", softmax_demo)
print("Probability sum:", softmax_demo.sum(axis=1))
```

ReLU: [[0. 0. 1.5]]

ReLU derivative: [[0. 0. 1.]]

Softmax: [[0.09003057 0.24472847 0.66524096]]

Probability sum: [1.]

## 17 18. Step 2 - Initialize the Network

The improved NumPy Deep Learning model keeps the A3 requirement (no external DL framework) but uses a stronger text representation:

```
Token IDs
  ↓
Embedding Layer
  ↓
Mean Pooling + Max Pooling
  ↓
Concatenate representation
  ↓
Dense: 128 → 256
  ↓ ReLU
Dense: 256 → 128
  ↓ ReLU
Dense: 128 → 30
  ↓
```

## Softmax

Improvements compared with the previous version:

- richer text representation using mean + max pooling;
- Adam optimizer implemented manually with NumPy;
- class-weighted cross entropy for imbalanced 30-category learning;
- gradient clipping for more stable training.

```
[38]: DL_EMBED_DIM = 64
DL_HIDDEN_1 = 256
DL_HIDDEN_2 = 128
DL_OUTPUT_DIM = len(CLASS_ORDER)

class_counts = np.bincount(y_train, minlength=DL_OUTPUT_DIM)
class_weights = len(y_train) / (DL_OUTPUT_DIM * np.maximum(class_counts, 1))
class_weights = np.clip(class_weights, 0.5, 8.0)

print("Class weight range:", class_weights.min(), class_weights.max())

class NumpyTextMLP:
    """
    Deep Learning from Scratch:
    Token IDs -> Embedding -> Mean+Max pooling -> Dense -> ReLU -> Dense ->
    ↪ Softmax

    Optimizer:
    Adam implemented manually with NumPy.
    """

    def __init__(self, vocab_size, embed_dim, hidden1, hidden2, output_dim,
    ↪ seed=42):
        rng = np.random.default_rng(seed)
        self.embed_dim = embed_dim
        self.embedding = rng.normal(
            0, 0.05, size=(vocab_size, embed_dim)
        )
        self.embedding[PAD_ID] = 0

        rep_dim = embed_dim * 2

        self.W1 = rng.normal(0, np.sqrt(2/rep_dim), (rep_dim, hidden1))
        self.b1 = np.zeros((1, hidden1))

        self.W2 = rng.normal(0, np.sqrt(2/hidden1), (hidden1, hidden2))
        self.b2 = np.zeros((1, hidden2))
```

```

self.W3 = rng.normal(0, np.sqrt(2/hidden2), (hidden2, output_dim))
self.b3 = np.zeros((1, output_dim))

self.params = [
    "embedding", "W1", "b1", "W2", "b2", "W3", "b3"
]

self.m = {p: np.zeros_like(getattr(self,p)) for p in self.params}
self.v = {p: np.zeros_like(getattr(self,p)) for p in self.params}
self.t = 0

def parameter_shapes(self):
    return {p:getattr(self,p).shape for p in self.params}

def forward(self, token_ids):

    token_ids = np.asarray(token_ids)

    emb = self.embedding[token_ids]

    mask = (token_ids != PAD_ID)

    lengths = np.maximum(mask.sum(axis=1, keepdims=True),1)

    mean_pool = (
        emb * mask[:, :, None]
    ).sum(axis=1) / lengths

    masked_emb = np.where(
        mask[:, :, None],
        emb,
        -1e9
    )

    max_pool = masked_emb.max(axis=1)

    pooled = np.concatenate(
        [mean_pool, max_pool],
        axis=1
    )

    z1 = pooled @ self.W1 + self.b1
    h1 = relu(z1)

    z2 = h1 @ self.W2 + self.b2

```

```

h2 = relu(z2)

z3 = h2 @ self.W3 + self.b3

prob = softmax(z3)

cache = {
    "token_ids":token_ids,
    "emb":emb,
    "mask":mask,
    "lengths":lengths,
    "mean_pool":mean_pool,
    "max_pool":max_pool,
    "pooled":pooled,
    "z1":z1,
    "h1":h1,
    "z2":z2,
    "h2":h2,
    "prob":prob,
}

return prob,cache

def cross_entropy(self,y_true,prob,sample_weights=None):

    y_true=np.asarray(y_true)

    losses=-np.log(
        np.clip(
            prob[np.arange(len(y_true)),y_true],
            1e-12,
            1
        )
    )

    if sample_weights is not None:
        losses*=sample_weights
        return losses.sum()/sample_weights.sum()

    return losses.mean()

def backward(self,y_true,cache,sample_weights=None):

    token_ids=cache["token_ids"]
    emb=cache["emb"]

```

```

mask=cache["mask"]
lengths=cache["lengths"]

prob=cache["prob"]
h1=cache["h1"]
h2=cache["h2"]

batch=len(y_true)

dZ3=prob.copy()
dZ3[np.arange(batch),y_true]==1

if sample_weights is not None:
    dZ3*=sample_weights[:,None]
    dZ3/=sample_weights.sum()
else:
    dZ3/=batch

grads={}

grads["W3"]=h2.T@dZ3
grads["b3"]=dZ3.sum(axis=0,keepdims=True)

dH2=dZ3@self.W3.T
dZ2=dH2*relu_derivative(cache["z2"])

grads["W2"]=h1.T@dZ2
grads["b2"]=dZ2.sum(axis=0,keepdims=True)

dH1=dZ2@self.W2.T
dZ1=dH1*relu_derivative(cache["z1"])

grads["W1"]=cache["pooled"].T@dZ1
grads["b1"]=dZ1.sum(axis=0,keepdims=True)

dPool=dZ1@self.W1.T

dMean=dPool[:,self.embed_dim]
dMax=dPool[:,self.embed_dim:]

dEmb=np.zeros_like(emb)

```

```

    # mean pooling gradient
    dEmb += (
        dMean[:,None,:]
        / lengths[:,None,:]
        * mask[:, :,None]
    )

    # max pooling gradient
    max_values=cache["max_pool"][:,None,:]

    max_mask=(
        emb==max_values
    ) & mask[:, :,None]

    dEmb += max_mask*dMax[:,None,:]

    dEmbedding=np.zeros_like(self.embedding)

    flat_ids=token_ids.reshape(-1)
    flat_grad=dEmb.reshape(-1,self.embed_dim)

    valid=flat_ids!=PAD_ID

    np.add.at(
        dEmbedding,
        flat_ids[valid],
        flat_grad[valid]
    )

    grads["embedding"]=dEmbedding

    # gradient clipping
    for k in grads:
        grads[k]=np.clip(grads[k],-5,5)

    return grads

def update(self,grads,lr=0.001):

    self.t+=1

    beta1=0.9
    beta2=0.999
    eps=1e-8

```

```

        for p in self.params:

            g=grads[p]

            self.m[p]=beta1*self.m[p]+(1-beta1)*g
            self.v[p]=beta2*self.v[p]+(1-beta2)*(g*g)

            m_hat=self.m[p]/(1-beta1**self.t)
            v_hat=self.v[p]/(1-beta2**self.t)

            param=getattr(self,p)

            param-=lr*m_hat/(np.sqrt(v_hat)+eps)

        self.embedding[PAD_ID]=0

    def predict_proba(self,x):
        return self.forward(x)[0]

    def predict(self,x):
        return np.argmax(self.predict_proba(x),axis=1)

model_demo=NumpyTextMLP(
    len(token_to_id),
    DL_EMBED_DIM,
    DL_HIDDEN_1,
    DL_HIDDEN_2,
    DL_OUTPUT_DIM,
    SEED
)

display(pd.DataFrame([
    {"parameter":k,"shape":v}
    for k,v in model_demo.parameter_shapes().items()
]))

```

Class weight range: 0.5 5.653909465020576

|   | parameter | shape      |
|---|-----------|------------|
| 0 | embedding | (4000, 64) |
| 1 | W1        | (128, 256) |
| 2 | b1        | (1, 256)   |
| 3 | W2        | (256, 128) |
| 4 | b2        | (1, 128)   |

|   |    |           |
|---|----|-----------|
| 5 | W3 | (128, 30) |
| 6 | b3 | (1, 30)   |

## 18 19. Step 3 - Forward Propagation

The forward pass applies the functions from input to output:

```

Token IDs
  ↓
Embedding lookup
  ↓
Mean pooling
  ↓
Z1 = H0 W1 + b1
  ↓
H1 = ReLU(Z1)
  ↓
Z2 = H1 W2 + b2
  ↓
H2 = ReLU(Z2)
  ↓
Z3 = H2 W3 + b3
  ↓
P = Softmax(Z3)

```

The intermediate values are stored in a cache because Step 5 backpropagation needs them.

### 18.0.1 Verification required

We explicitly check: - output shape - each probability row sums to approximately 1 - no NaN/Inf values are present

```

[39]: demo_ids_batch = X_train_ids[:4]
demo_model = NumpyTextMLP(
    vocab_size=len(token_to_id),
    embed_dim=DL_EMBED_DIM,
    hidden1=DL_HIDDEN_1,
    hidden2=DL_HIDDEN_2,
    output_dim=DL_OUTPUT_DIM,
    seed=SEED,
)

demo_prob, demo_cache = demo_model.forward(demo_ids_batch)

print("Input token-ID shape:", demo_ids_batch.shape)
print("Embedding batch shape:", demo_cache["emb"].shape)
print("Mean-pooled shape:", demo_cache["pooled"].shape)
print("Output probability shape:", demo_prob.shape)
print("Probability sums:", np.round(demo_prob.sum(axis=1), 6))

```

```
print("Contains NaN:", np.isnan(demo_prob).any())
print("Contains Inf:", np.isinf(demo_prob).any())
```

```
Input token-ID shape: (4, 40)
Embedding batch shape: (4, 40, 64)
Mean-pooled shape: (4, 128)
Output probability shape: (4, 30)
Probability sums: [1. 1. 1. 1.]
Contains NaN: False
Contains Inf: False
```

## 19 20. Step 4 - Cross-Entropy Loss

For one review:

Loss =  $-\log(P(\text{correct class}))$

For the mini-batch:

$L = -(1/N) \sum \log(P_{i,y_i})$

A correct high-probability prediction produces a small loss; assigning a small probability to the true class produces a larger loss.

### 19.0.1 Verification required

We compute the loss on the small forward-pass example before training.

```
[40]: demo_y = y_train[:4]

demo_loss = demo_model.cross_entropy(demo_y, demo_prob)

print("True class IDs:", demo_y)
print("Predicted probabilities for the true classes:")
print(np.round(demo_prob[np.arange(len(demo_y)), demo_y], 6))
print("Example cross-entropy loss:", round(float(demo_loss), 6))
```

```
True class IDs: [ 0 22 11  0]
Predicted probabilities for the true classes:
[0.03357  0.036651 0.037517 0.03393 ]
Example cross-entropy loss: 3.341709
```

## 20 21. Step 5 - Backpropagation

Backpropagation is the systematic application of the chain rule.

The lecture's core idea is:

```
Loss
↓
output layer
```

$\downarrow$   
 hidden layer 2  
 $\downarrow$   
 hidden layer 1  
 $\downarrow$   
 input representation

For Softmax + cross-entropy:

$$dZ3 = (P - Y) / N$$

Then:

$$\begin{aligned}
 dW3 &= H2.T \ dZ3 \\
 dH2 &= dZ3 \ W3.T \\
 dZ2 &= dH2 \ \text{ReLU}'(Z2) \\
 dW2 &= H1.T \ dZ2 \\
 dH1 &= dZ2 \ W2.T \\
 dZ1 &= dH1 \ \text{ReLU}'(Z1) \\
 dW1 &= H0.T \ dZ1
 \end{aligned}$$

For this text model, the gradient also continues into the embedding representation through the mean-pooling operation.

### 20.0.1 Verification required

The next cell prints the shape of every gradient tensor.

```
[41]: demo_gradients = demo_model.backward(demo_y, demo_cache)

for name, value in demo_gradients.items():
    print(f"{name:12s}: {value.shape} | finite={np.isfinite(value).all()}")

W3           : (128, 30) | finite=True
b3           : (1, 30) | finite=True
W2           : (256, 128) | finite=True
b2           : (1, 128) | finite=True
W1           : (128, 256) | finite=True
b1           : (1, 256) | finite=True
embedding    : (4000, 64) | finite=True
```

## 21 22. Step 6 - Gradient Descent

Once the gradients are known, every trainable parameter is updated:

$$\begin{aligned}
 W &\leftarrow W - dW \\
 b &\leftarrow b - db \\
 \text{Embedding} &\leftarrow \text{Embedding} - d\text{Embedding}
 \end{aligned}$$

This is the same optimization principle as the lecture's dense-network example.

### 21.0.1 Verification required

We perform one real gradient update on the small demonstration batch and compare the loss before and after the update.

```
[42]: update_model = NumpyTextMLP(
        vocab_size=len(token_to_id),
        embed_dim=DL_EMBED_DIM,
        hidden1=DL_HIDDEN_1,
        hidden2=DL_HIDDEN_2,
        output_dim=DL_OUTPUT_DIM,
        seed=SEED,
    )

    prob_before, cache_before = update_model.forward(demo_ids_batch)
    loss_before = update_model.cross_entropy(demo_y, prob_before)

    grads_before = update_model.backward(demo_y, cache_before)

    LEARNING_RATE_CHECK = 0.02
    update_model.update(grads_before, LEARNING_RATE_CHECK)

    prob_after, _ = update_model.forward(demo_ids_batch)
    loss_after = update_model.cross_entropy(demo_y, prob_after)

    print("Loss before one update:", round(loss_before, 6))
    print("Loss after one update:", round(loss_after, 6))
    print("Loss change:", round(loss_after - loss_before, 6))
```

Loss before one update: 3.341709

Loss after one update: 1.524595

Loss change: -1.817114

## 22 23. Step 7 - Training Loop

The complete training cycle is:

```
Forward
  ↓
Cross-Entropy Loss
  ↓
Backpropagation
  ↓
Gradient Descent
  ↓
Repeat
```

A mini-batch loop is used because the full text dataset is larger than the small lecture example.

Validation loss is recorded after every epoch:

Training loss           → learning on seen data  
Validation loss       → generalization on unseen data

The test set remains untouched until Step 9.

```
[43]: DL_BATCH_SIZE = 256
DL_EPOCHS = 100
DL_LEARNING_RATE = 0.001

dl_model = NumpyTextMLP(
    vocab_size=len(token_to_id),
    embed_dim=DL_EMBED_DIM,
    hidden1=DL_HIDDEN_1,
    hidden2=DL_HIDDEN_2,
    output_dim=DL_OUTPUT_DIM,
    seed=SEED,
)

train_loss_history=[]
val_loss_history=[]

num_train=len(X_train_ids)

best_val_loss=np.inf
patience=15
wait=0

for epoch in range(DL_EPOCHS):

    rng=np.random.default_rng(SEED+epoch)
    indices=rng.permutation(num_train)

    for start in range(0,num_train,DL_BATCH_SIZE):

        idx=indices[start:start+DL_BATCH_SIZE]

        xb=X_train_ids[idx]
        yb=y_train[idx]

        prob,cache=dl_model.forward(xb)

        weights=class_weights[yb]

        loss=dl_model.cross_entropy(
            yb,
            prob,
            weights
        )
```

```

        grads=dl_model.backward(
            yb,
            cache,
            weights
        )

        dl_model.update(
            grads,
            DL_LEARNING_RATE
        )

    train_prob=dl_model.predict_proba(
        X_train_ids[:5000]
    )

    val_prob=dl_model.predict_proba(
        X_val_ids
    )

    train_loss=dl_model.cross_entropy(
        y_train[:5000],
        train_prob
    )

    val_loss=dl_model.cross_entropy(
        y_val,
        val_prob
    )

    train_loss_history.append(train_loss)
    val_loss_history.append(val_loss)

    print(
        f"Epoch {epoch+1:03d}/{DL_EPOCHS} "
        f"Train Loss={train_loss:.4f} "
        f"Val Loss={val_loss:.4f}"
    )

    if val_loss < best_val_loss:
        best_val_loss=val_loss
        wait=0
    else:
        wait+=1

```

|               |                   |                 |
|---------------|-------------------|-----------------|
| Epoch 001/100 | Train Loss=2.1934 | Val Loss=2.2236 |
| Epoch 002/100 | Train Loss=1.1792 | Val Loss=1.3107 |
| Epoch 003/100 | Train Loss=0.9229 | Val Loss=1.1278 |
| Epoch 004/100 | Train Loss=0.8334 | Val Loss=1.1100 |
| Epoch 005/100 | Train Loss=0.7539 | Val Loss=1.1019 |
| Epoch 006/100 | Train Loss=0.6983 | Val Loss=1.1237 |
| Epoch 007/100 | Train Loss=0.6218 | Val Loss=1.1154 |
| Epoch 008/100 | Train Loss=0.5936 | Val Loss=1.1654 |
| Epoch 009/100 | Train Loss=0.5156 | Val Loss=1.1467 |
| Epoch 010/100 | Train Loss=0.5051 | Val Loss=1.2107 |
| Epoch 011/100 | Train Loss=0.4540 | Val Loss=1.2245 |
| Epoch 012/100 | Train Loss=0.4325 | Val Loss=1.2691 |
| Epoch 013/100 | Train Loss=0.4040 | Val Loss=1.3078 |
| Epoch 014/100 | Train Loss=0.3973 | Val Loss=1.3556 |
| Epoch 015/100 | Train Loss=0.3510 | Val Loss=1.3749 |
| Epoch 016/100 | Train Loss=0.3392 | Val Loss=1.4344 |
| Epoch 017/100 | Train Loss=0.3304 | Val Loss=1.4867 |
| Epoch 018/100 | Train Loss=0.3098 | Val Loss=1.5126 |
| Epoch 019/100 | Train Loss=0.3111 | Val Loss=1.5748 |
| Epoch 020/100 | Train Loss=0.2914 | Val Loss=1.6149 |
| Epoch 021/100 | Train Loss=0.2708 | Val Loss=1.6410 |
| Epoch 022/100 | Train Loss=0.2785 | Val Loss=1.7223 |
| Epoch 023/100 | Train Loss=0.2555 | Val Loss=1.7338 |
| Epoch 024/100 | Train Loss=0.2635 | Val Loss=1.7896 |
| Epoch 025/100 | Train Loss=0.2631 | Val Loss=1.8356 |
| Epoch 026/100 | Train Loss=0.2246 | Val Loss=1.8418 |
| Epoch 027/100 | Train Loss=0.2198 | Val Loss=1.8832 |
| Epoch 028/100 | Train Loss=0.2155 | Val Loss=1.9353 |
| Epoch 029/100 | Train Loss=0.2218 | Val Loss=1.9827 |
| Epoch 030/100 | Train Loss=0.2100 | Val Loss=2.0151 |
| Epoch 031/100 | Train Loss=0.2214 | Val Loss=2.0669 |
| Epoch 032/100 | Train Loss=0.2003 | Val Loss=2.0920 |
| Epoch 033/100 | Train Loss=0.2000 | Val Loss=2.1297 |
| Epoch 034/100 | Train Loss=0.2070 | Val Loss=2.1803 |
| Epoch 035/100 | Train Loss=0.1929 | Val Loss=2.1715 |
| Epoch 036/100 | Train Loss=0.1911 | Val Loss=2.2186 |
| Epoch 037/100 | Train Loss=0.2004 | Val Loss=2.2550 |
| Epoch 038/100 | Train Loss=0.1922 | Val Loss=2.3088 |
| Epoch 039/100 | Train Loss=0.1794 | Val Loss=2.3216 |
| Epoch 040/100 | Train Loss=0.2012 | Val Loss=2.4130 |
| Epoch 041/100 | Train Loss=0.1750 | Val Loss=2.3827 |
| Epoch 042/100 | Train Loss=0.1703 | Val Loss=2.4327 |
| Epoch 043/100 | Train Loss=0.1624 | Val Loss=2.4303 |
| Epoch 044/100 | Train Loss=0.1676 | Val Loss=2.4904 |
| Epoch 045/100 | Train Loss=0.1621 | Val Loss=2.4898 |
| Epoch 046/100 | Train Loss=0.1572 | Val Loss=2.5349 |
| Epoch 047/100 | Train Loss=0.1687 | Val Loss=2.5859 |
| Epoch 048/100 | Train Loss=0.1596 | Val Loss=2.6160 |

Epoch 049/100 Train Loss=0.1525 Val Loss=2.6142  
Epoch 050/100 Train Loss=0.1419 Val Loss=2.6862  
Epoch 051/100 Train Loss=0.1564 Val Loss=2.7190  
Epoch 052/100 Train Loss=0.1436 Val Loss=2.7422  
Epoch 053/100 Train Loss=0.1494 Val Loss=2.7531  
Epoch 054/100 Train Loss=0.1413 Val Loss=2.7591  
Epoch 055/100 Train Loss=0.1437 Val Loss=2.8323  
Epoch 056/100 Train Loss=0.1471 Val Loss=2.8555  
Epoch 057/100 Train Loss=0.1302 Val Loss=2.8490  
Epoch 058/100 Train Loss=0.1361 Val Loss=2.8643  
Epoch 059/100 Train Loss=0.1353 Val Loss=2.9348  
Epoch 060/100 Train Loss=0.1411 Val Loss=2.9660  
Epoch 061/100 Train Loss=0.1372 Val Loss=3.0073  
Epoch 062/100 Train Loss=0.1368 Val Loss=2.9880  
Epoch 063/100 Train Loss=0.1213 Val Loss=3.0091  
Epoch 064/100 Train Loss=0.1388 Val Loss=3.0577  
Epoch 065/100 Train Loss=0.1218 Val Loss=3.0600  
Epoch 066/100 Train Loss=0.1271 Val Loss=3.1207  
Epoch 067/100 Train Loss=0.1195 Val Loss=3.1199  
Epoch 068/100 Train Loss=0.1138 Val Loss=3.1376  
Epoch 069/100 Train Loss=0.1151 Val Loss=3.1592  
Epoch 070/100 Train Loss=0.1131 Val Loss=3.2444  
Epoch 071/100 Train Loss=0.1199 Val Loss=3.2580  
Epoch 072/100 Train Loss=0.1111 Val Loss=3.2581  
Epoch 073/100 Train Loss=0.1160 Val Loss=3.2785  
Epoch 074/100 Train Loss=0.1233 Val Loss=3.3062  
Epoch 075/100 Train Loss=0.1072 Val Loss=3.3523  
Epoch 076/100 Train Loss=0.1387 Val Loss=3.4610  
Epoch 077/100 Train Loss=0.1107 Val Loss=3.3630  
Epoch 078/100 Train Loss=0.1132 Val Loss=3.4472  
Epoch 079/100 Train Loss=0.1030 Val Loss=3.3975  
Epoch 080/100 Train Loss=0.1115 Val Loss=3.4337  
Epoch 081/100 Train Loss=0.1062 Val Loss=3.4411  
Epoch 082/100 Train Loss=0.1001 Val Loss=3.4685  
Epoch 083/100 Train Loss=0.1012 Val Loss=3.4743  
Epoch 084/100 Train Loss=0.1102 Val Loss=3.5309  
Epoch 085/100 Train Loss=0.1006 Val Loss=3.5144  
Epoch 086/100 Train Loss=0.0937 Val Loss=3.5269  
Epoch 087/100 Train Loss=0.0963 Val Loss=3.5434  
Epoch 088/100 Train Loss=0.0983 Val Loss=3.6008  
Epoch 089/100 Train Loss=0.0891 Val Loss=3.5886  
Epoch 090/100 Train Loss=0.1010 Val Loss=3.6545  
Epoch 091/100 Train Loss=0.0947 Val Loss=3.6030  
Epoch 092/100 Train Loss=0.0944 Val Loss=3.6564  
Epoch 093/100 Train Loss=0.0999 Val Loss=3.6768  
Epoch 094/100 Train Loss=0.0973 Val Loss=3.6879  
Epoch 095/100 Train Loss=0.0923 Val Loss=3.6944  
Epoch 096/100 Train Loss=0.0922 Val Loss=3.7317

```
Epoch 097/100 Train Loss=0.0839 Val Loss=3.7559
Epoch 098/100 Train Loss=0.0800 Val Loss=3.7342
Epoch 099/100 Train Loss=0.0813 Val Loss=3.8010
Epoch 100/100 Train Loss=0.0823 Val Loss=3.8090
```

### 22.0.1 Training Curve

The expected pattern is:

- training loss should decrease as the network learns;
- validation loss should decrease while generalization improves;
- a widening train/validation gap can indicate overfitting.

The exact conclusion must be based on the curve produced by the executed notebook.

## 23 24. Step 8 - Prediction

After training, the network returns:

```
P(class 0), P(class 1), ..., P(class 29)
```

The class with the largest probability becomes the predicted interest category.

Unlike the SVM score visualization, the neural network's Softmax outputs are proper class probabilities because they are normalized to sum to 1.

The test set is used here for the first time in the Deep Learning section.

```
[44]: final_dl_prob = dl_model.predict_proba(X_test_ids)
      final_dl_pred = np.argmax(final_dl_prob, axis=1)

      print("Prediction shape:", final_dl_pred.shape)
      print("Probability matrix shape:", final_dl_prob.shape)

      print("\nFirst 5 test predictions:")
      sample_pred_df = pd.DataFrame({
          "Actual": [ID_TO_CLASS[i] for i in y_test[:5]],
          "Predicted": [ID_TO_CLASS[i] for i in final_dl_pred[:5]],
          "Confidence": np.max(final_dl_prob[:5], axis=1).round(4),
      })

      display(sample_pred_df)

      print("Probability sums for first 5 rows:")
      print(np.round(final_dl_prob[:5].sum(axis=1), 6))
```

```
Prediction shape: (5889,)
```

```
Probability matrix shape: (5889, 30)
```

```
First 5 test predictions:
```

|   | Actual                    | Predicted                 | Confidence |
|---|---------------------------|---------------------------|------------|
| 0 | Cell Phones & Smartphones | Motherboards              | 0.2213     |
| 1 | Cell Phones & Smartphones | Cell Phones & Smartphones | 1.0000     |
| 2 | Cell Phones & Smartphones | Ink Cartridges            | 0.7625     |
| 3 | Video Games               | Video Games               | 0.9999     |
| 4 | Headphones                | Keyboards & Keypads       | 0.9998     |

Probability sums for first 5 rows:  
[1. 1. 1. 1. 1.]

## 24 25. Step 9 - Evaluation

The final Deep Learning model is evaluated on the same untouched test set as the three traditional ML models.

Metrics:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1
- Balanced Accuracy
- One-vs-Rest ROC-AUC

A confusion matrix is also shown.

The purpose is to make a fair, simultaneous four-model comparison possible.

```
[45]: final_dl_metrics = classification_metrics(
    y_test,
    final_dl_pred,
    final_dl_prob,
)

final_dl_metrics["Model"] = "Deep Learning - NumPy"
final_dl_metrics["Training_Time_sec"] = np.nan
final_dl_metrics["Final_Training_Loss"] = train_loss_history[-1]

display(pd.DataFrame([final_dl_metrics]))

cm_dl = confusion_matrix(
    y_test,
    final_dl_pred,
    labels=np.arange(len(CLASS_ORDER)),
)

plt.figure(figsize=(12, 10))
sns.heatmap(
    cm_dl,
    annot=False,
```

```

    cmap="Greens",
    xticklabels=CLASS_ORDER,
    yticklabels=CLASS_ORDER,
)

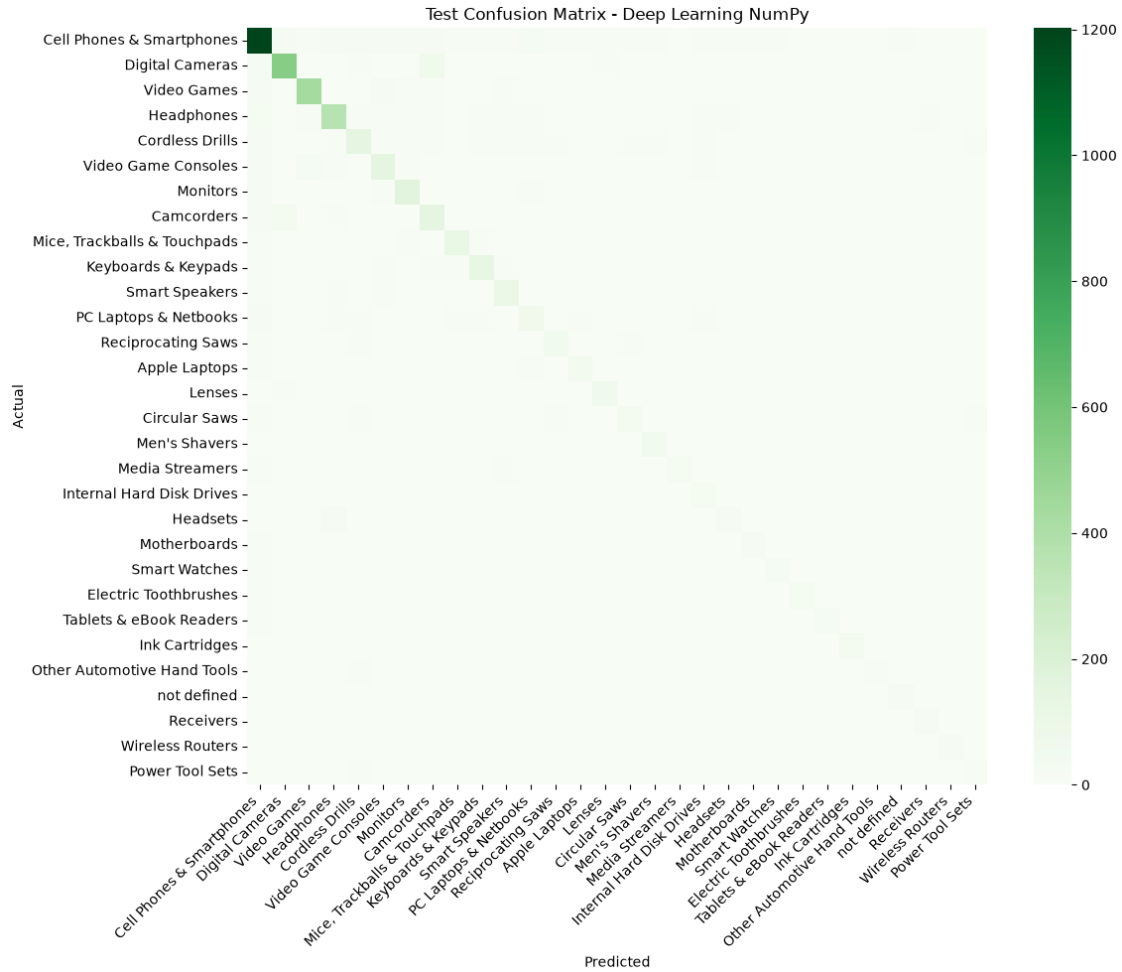
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Test Confusion Matrix - Deep Learning NumPy")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig(FIG_DIR / "12_dl_confusion_matrix.png", dpi=180)
plt.show()

```

|   | Accuracy | Precision_macro | Recall_macro | F1_macro | Balanced_Accuracy | \ |
|---|----------|-----------------|--------------|----------|-------------------|---|
| 0 | 0.688232 | 0.55426         | 0.562831     | 0.555029 | 0.562831          |   |

|   | ROC_AUC_OVR_macro | Model                 | Training_Time_sec | \ |
|---|-------------------|-----------------------|-------------------|---|
| 0 | 0.93952           | Deep Learning - NumPy | NaN               |   |

|   | Final_Training_Loss |
|---|---------------------|
| 0 | 0.082291            |



## 25 26. Final Comparison - All Four Models Together

This is the central A3 comparison.

The four models are evaluated **together**:

1. Logistic Regression
2. Linear SVM
3. Multinomial Naive Bayes
4. Deep Learning - NumPy

There is no two-stage comparison where a “best ML model” is compared against Deep Learning after the fact. All four are placed in the same final table and visualization.

All four use the same held-out test set and the same core metrics.

```
[46]: final_four = pd.concat([
      traditional_results[[
```

```

        "Model",
        "Accuracy",
        "Precision_macro",
        "Recall_macro",
        "F1_macro",
        "Balanced_Accuracy",
        "ROC_AUC_OVR_macro",
        "Training_Time_sec",
    ]],
    pd.DataFrame([
        {
            "Model": "Deep Learning - NumPy",
            "Accuracy": final_dl_metrics["Accuracy"],
            "Precision_macro": final_dl_metrics["Precision_macro"],
            "Recall_macro": final_dl_metrics["Recall_macro"],
            "F1_macro": final_dl_metrics["F1_macro"],
            "Balanced_Accuracy": final_dl_metrics["Balanced_Accuracy"],
            "ROC_AUC_OVR_macro": final_dl_metrics["ROC_AUC_OVR_macro"],
            "Training_Time_sec": np.nan,
        }
    ]),
    ignore_index=True)

final_four = final_four.sort_values(
    "F1_macro",
    ascending=False,
).reset_index(drop=True)

display(final_four)

```

|   | Model                   | Accuracy | Precision_macro | Recall_macro | F1_macro | \ |
|---|-------------------------|----------|-----------------|--------------|----------|---|
| 0 | Linear SVM              | 0.776193 | 0.781943        | 0.629196     | 0.690231 |   |
| 1 | Logistic Regression     | 0.771268 | 0.854291        | 0.596947     | 0.686306 |   |
| 2 | Deep Learning - NumPy   | 0.688232 | 0.554260        | 0.562831     | 0.555029 |   |
| 3 | Multinomial Naive Bayes | 0.688232 | 0.888586        | 0.391919     | 0.483426 |   |

|   | Balanced_Accuracy | ROC_AUC_OVR_macro | Training_Time_sec |
|---|-------------------|-------------------|-------------------|
| 0 | 0.629196          | 0.944697          | 10.740268         |
| 1 | 0.596947          | 0.957819          | 7.488329          |
| 2 | 0.562831          | 0.939520          | NaN               |
| 3 | 0.391919          | 0.914931          | 0.050471          |

### 25.0.1 How the final model comparison should be interpreted

Macro F1 is used as the main ranking metric, but the final decision should not be made from one number alone.

We examine:

- F1 for overall multi-class balance
- Recall for missed interests

- Accuracy for overall correctness
- ROC-AUC for ranking/discrimination where meaningful
- Training time as a deployment consideration

A model can be the strongest predictive model without being the cheapest model to deploy.

```
[47]: comparison_melt = final_four.melt(
    id_vars="Model",
    value_vars=[
        "Accuracy",
        "Precision_macro",
        "Recall_macro",
        "F1_macro",
    ],
    var_name="Metric",
    value_name="Score",
)

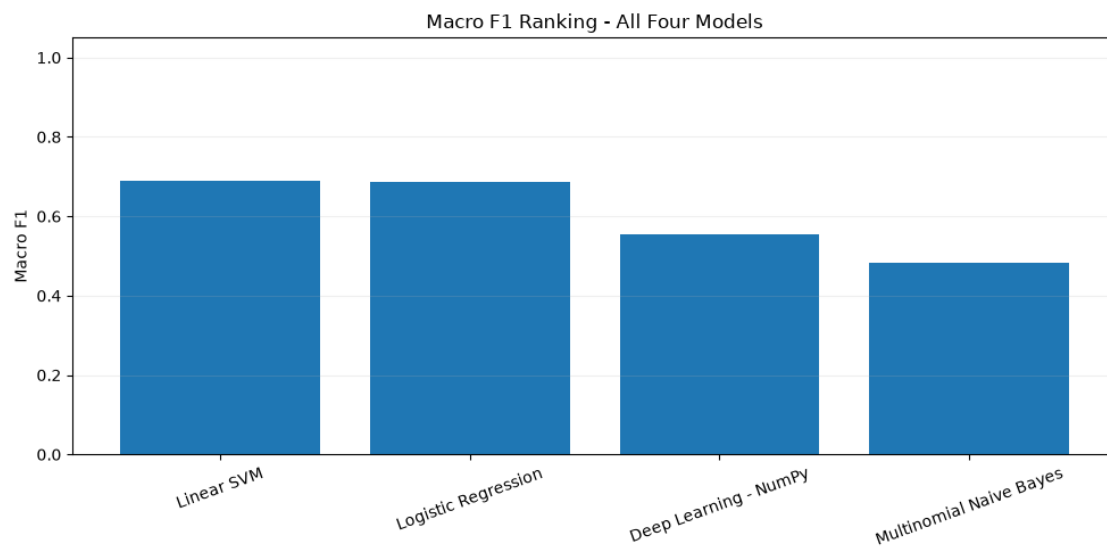
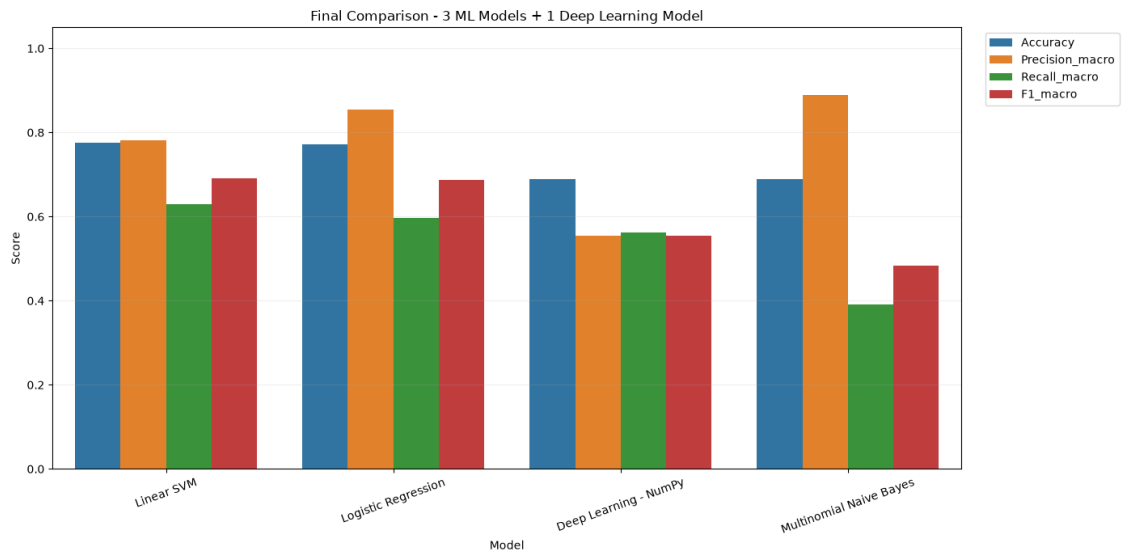
plt.figure(figsize=(14, 7))
sns.barplot(
    data=comparison_melt,
    x="Model",
    y="Score",
    hue="Metric",
)

plt.ylim(0, 1.05)
plt.title("Final Comparison - 3 ML Models + 1 Deep Learning Model")
plt.xticks(rotation=20)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "13_four_model_comparison.png", dpi=180)
plt.show()

plt.figure(figsize=(10, 5))
plt.bar(
    final_four["Model"],
    final_four["F1_macro"],
)

plt.ylim(0, 1.05)
plt.title("Macro F1 Ranking - All Four Models")
plt.ylabel("Macro F1")
plt.xticks(rotation=20)
plt.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "14_four_model_macro_f1_ranking.png", dpi=180)
```

```
plt.show()
```



```
[48]: metric_heatmap = final_four.set_index("Model") [  
    [  
        "Accuracy",  
        "Precision_macro",  
        "Recall_macro",  
        "F1_macro",  
        "Balanced_Accuracy",  
        "ROC_AUC_OVR_macro",  
    ]  
]
```

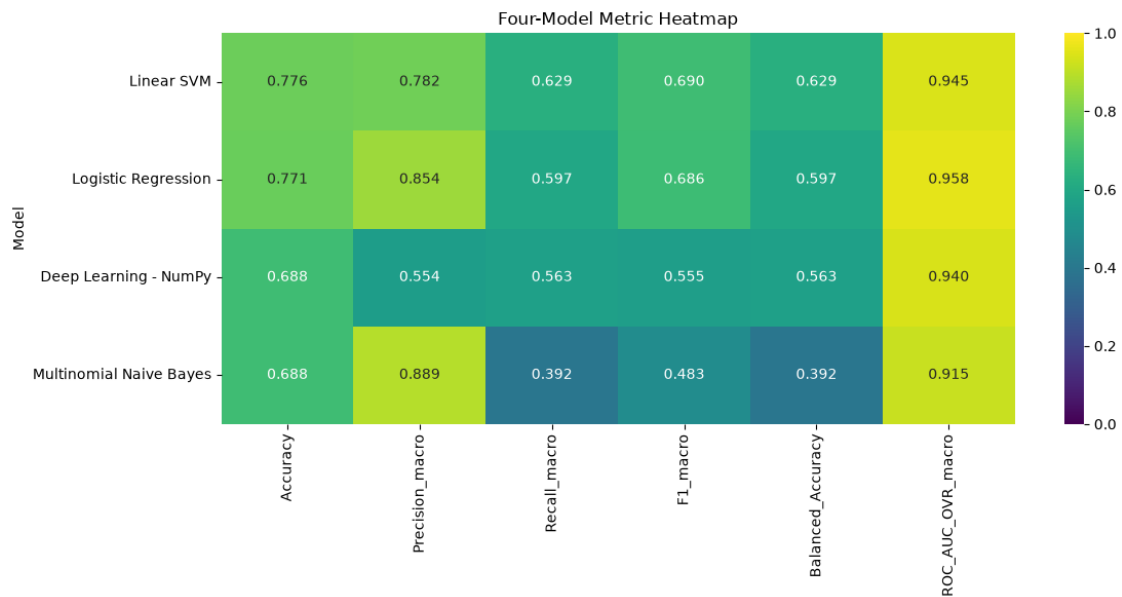
```

]
]

plt.figure(figsize=(12, 6))
sns.heatmap(
    metric_heatmap,
    annot=True,
    fmt=".3f",
    cmap="viridis",
    vmin=0,
    vmax=1,
)

plt.title("Four-Model Metric Heatmap")
plt.tight_layout()
plt.savefig(FIG_DIR / "15_four_model_metric_heatmap.png", dpi=180)
plt.show()

```



```

[49]: # Actual vs predicted class agreement by model.
prediction_table = pd.DataFrame({
    "Actual": y_test,
    "Logistic Regression": ml_predictions["Logistic Regression"],
    "Linear SVM": ml_predictions["Linear SVM"],
    "Multinomial Naive Bayes": ml_predictions["Multinomial Naive Bayes"],
    "Deep Learning - NumPy": final_dl_pred,
})

```

```

agreement_rows = []

for model_name in prediction_table.columns[1:]:
    agreement_rows.append({
        "Model": model_name,
        "Correct": int((prediction_table["Actual"] ==
↪ prediction_table[model_name]).sum()),
        "Incorrect": int((prediction_table["Actual"] !=
↪ prediction_table[model_name]).sum()),
    })

agreement_df = pd.DataFrame(agreement_rows)
display(agreement_df)

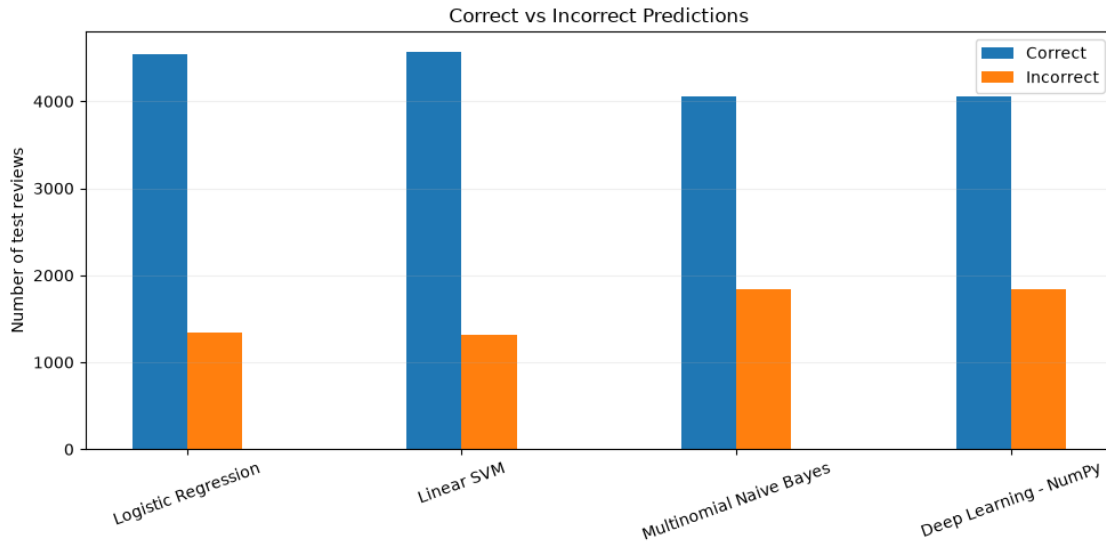
plt.figure(figsize=(10, 5))
width = 0.2
x = np.arange(len(agreement_df))

plt.bar(x - 1.5 * width, agreement_df["Correct"], width, label="Correct")
plt.bar(x - 0.5 * width, agreement_df["Incorrect"], width, label="Incorrect")

plt.xticks(x - width, agreement_df["Model"], rotation=20)
plt.ylabel("Number of test reviews")
plt.title("Correct vs Incorrect Predictions")
plt.legend()
plt.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "16_correct_incorrect_predictions.png", dpi=180)
plt.show()

```

|   | Model                   | Correct | Incorrect |
|---|-------------------------|---------|-----------|
| 0 | Logistic Regression     | 4542    | 1347      |
| 1 | Linear SVM              | 4571    | 1318      |
| 2 | Multinomial Naive Bayes | 4053    | 1836      |
| 3 | Deep Learning - NumPy   | 4053    | 1836      |



### 25.0.2 Confusion-Matrix Comparison

The four confusion matrices use the same class order and the same test set.

This visualization is intentionally included because category-level accuracy can hide which product interests are being confused.

```
[50]: all_prediction_maps = {
    "Logistic Regression": ml_predictions["Logistic Regression"],
    "Linear SVM": ml_predictions["Linear SVM"],
    "Multinomial Naive Bayes": ml_predictions["Multinomial Naive Bayes"],
    "Deep Learning - NumPy": final_dl_pred,
}

fig, axes = plt.subplots(2, 2, figsize=(18, 16))
axes = axes.ravel()

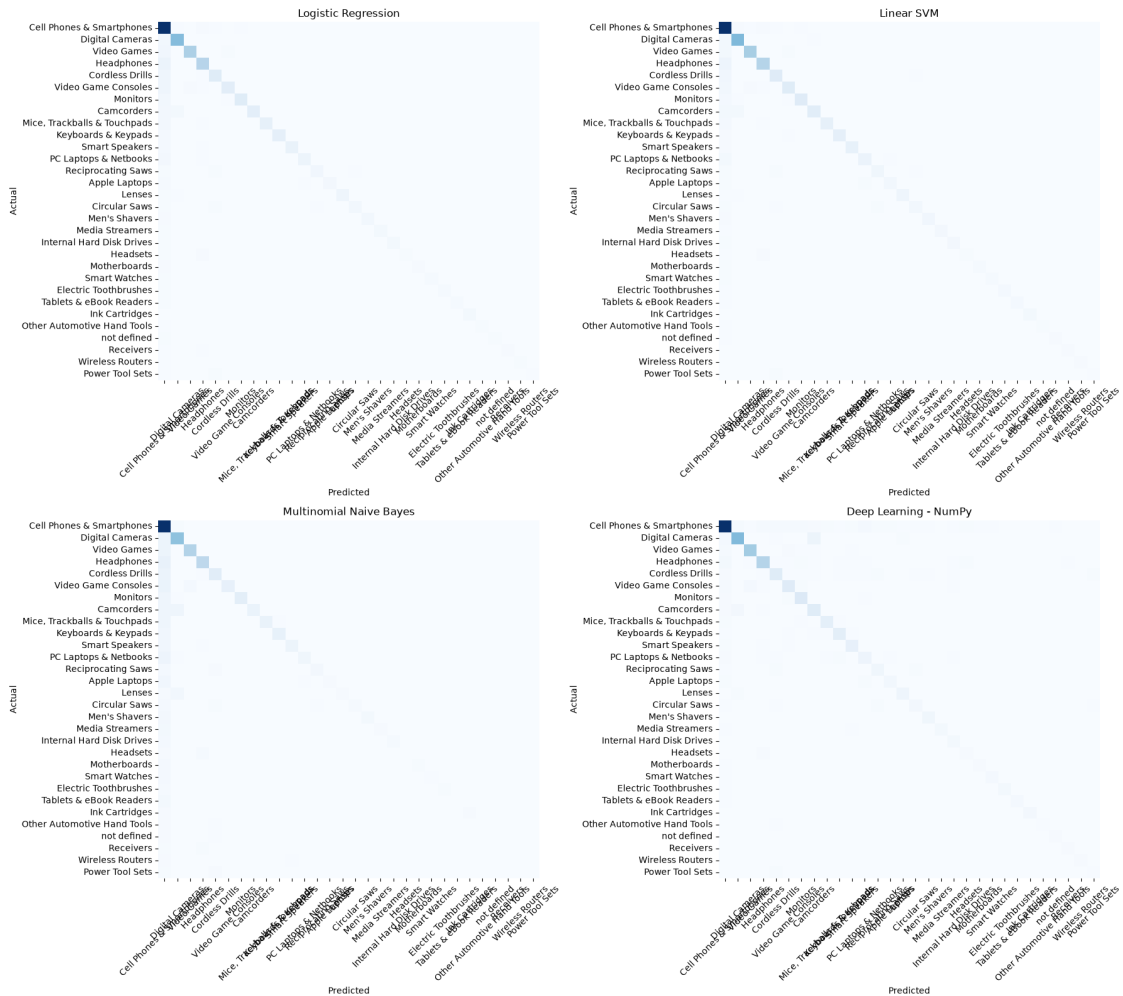
for ax, (name, pred) in zip(axes, all_prediction_maps.items()):
    cm = confusion_matrix(y_test, pred, labels=np.arange(len(CLASS_ORDER)))
    sns.heatmap(
        cm,
        annot=False,
        cmap="Blues",
        cbar=False,
        xticklabels=CLASS_ORDER,
        yticklabels=CLASS_ORDER,
        ax=ax,
    )
    ax.set_title(name)
```

```

ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.tick_params(axis="x", rotation=45)
ax.tick_params(axis="y", rotation=0)

plt.tight_layout()
plt.savefig(FIG_DIR / "17_all_four_confusion_matrices.png", dpi=180)
plt.show()

```



## 26 27. Probability / Confidence Visualization

For deployment, it is useful to show not only the predicted class but also model confidence.

- Logistic Regression and Naive Bayes provide class probabilities.
- Deep Learning Softmax provides class probabilities.
- Linear SVM provides decision scores rather than calibrated probabilities; the notebook uses

normalized scores only for visualization and explicitly does not call them calibrated probabilities.

The next cell compares the top predicted confidence for the best available deployed models.

```
[51]: def get_model_probabilities(model_name):
    if model_name == "Deep Learning - NumPy":
        return final_dl_prob, True

    if model_name == "Linear SVM":
        return ml_probabilities["Linear SVM (normalized scores)", False]

    return ml_probabilities[model_name], True

probability_models = [
    "Logistic Regression",
    "Multinomial Naive Bayes",
    "Deep Learning - NumPy",
]

confidence_df = []

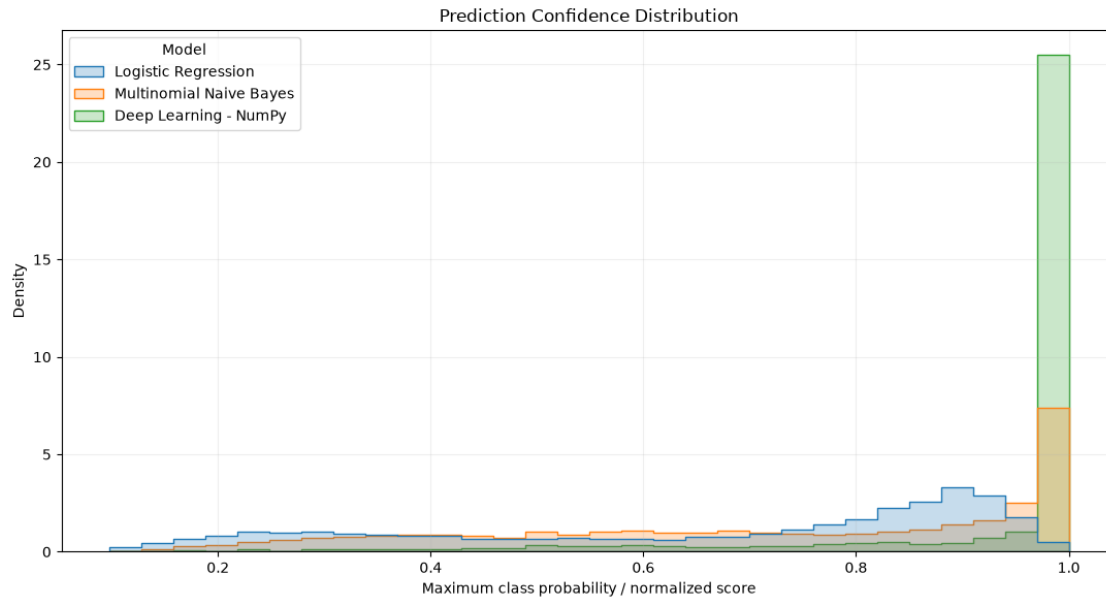
for model_name in probability_models:
    prob, is_probability = get_model_probabilities(model_name)
    confidence_df.append(
        pd.DataFrame({
            "Model": model_name,
            "Confidence": np.max(prob, axis=1),
            "Is calibrated probability": is_probability,
        })
    )

confidence_df = pd.concat(confidence_df, ignore_index=True)

plt.figure(figsize=(11, 6))
sns.histplot(
    data=confidence_df,
    x="Confidence",
    hue="Model",
    bins=30,
    element="step",
    stat="density",
    common_norm=False,
)

plt.title("Prediction Confidence Distribution")
plt.xlabel("Maximum class probability / normalized score")
plt.ylabel("Density")
```

```
plt.grid(alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "18_confidence_distribution.png", dpi=180)
plt.show()
```



## 27 28. Error Analysis

A strong AI workflow does not stop at aggregate metrics.

For text classification, common failure modes include:

- semantically similar product categories;
- reviews that mention several product types;
- very short reviews with little discriminative vocabulary;
- reviews assigned to **Other** because the raw category is rare;
- highly positive/negative language that carries sentiment but not category identity.

The following cells identify the most common actual → predicted confusions and show example misclassified reviews.

```
[52]: best_four_name = final_four.iloc[0]["Model"]
best_four_pred = all_prediction_maps[best_four_name]

error_pairs = (
    pd.DataFrame({
        "Actual": [ID_TO_CLASS[i] for i in y_test],
        "Predicted": [ID_TO_CLASS[i] for i in best_four_pred],
    })
)
```

```

        .query("Actual != Predicted")
        .value_counts(["Actual", "Predicted"])
        .reset_index(name="count")
        .head(20)
    )

display(error_pairs)

```

|    | Actual                       | Predicted                 | count |
|----|------------------------------|---------------------------|-------|
| 0  | Headphones                   | Cell Phones & Smartphones | 72    |
| 1  | Cordless Drills              | Cell Phones & Smartphones | 50    |
| 2  | Video Game Consoles          | Cell Phones & Smartphones | 47    |
| 3  | Video Games                  | Cell Phones & Smartphones | 36    |
| 4  | PC Laptops & Netbooks        | Cell Phones & Smartphones | 34    |
| 5  | Camcorders                   | Cell Phones & Smartphones | 33    |
| 6  | Camcorders                   | Digital Cameras           | 33    |
| 7  | Digital Cameras              | Cell Phones & Smartphones | 33    |
| 8  | Monitors                     | Cell Phones & Smartphones | 32    |
| 9  | Mice, Trackballs & Touchpads | Cell Phones & Smartphones | 28    |
| 10 | Keyboards & Key pads         | Cell Phones & Smartphones | 21    |
| 11 | Cell Phones & Smartphones    | Headphones                | 20    |
| 12 | Headsets                     | Headphones                | 20    |
| 13 | Men's Shavers                | Cell Phones & Smartphones | 19    |
| 14 | Smart Speakers               | Cell Phones & Smartphones | 19    |
| 15 | Video Game Consoles          | Video Games               | 18    |
| 16 | Cell Phones & Smartphones    | Digital Cameras           | 18    |
| 17 | Video Games                  | Video Game Consoles       | 16    |
| 18 | Apple Laptops                | Cell Phones & Smartphones | 16    |
| 19 | Motherboards                 | Cell Phones & Smartphones | 15    |

```

[53]: misclassified_idx = np.where(y_test != best_four_pred)[0][:12]

error_examples = test_df.iloc[misclassified_idx][
    ["category", "review title", "review content", "rating",
    ↪ "interest_category"]
].copy()

error_examples["Predicted"] = [ID_TO_CLASS[i] for i in
    ↪ best_four_pred[misclassified_idx]]

display(error_examples)

```

|       | category \                   |
|-------|------------------------------|
| 37765 | Cell Phones & Smartphones    |
| 1561  | Video Game Consoles          |
| 20403 | Mice, Trackballs & Touchpads |
| 40541 | Headphones                   |
| 31419 | Cell Phones & Smartphones    |

|       |                              |
|-------|------------------------------|
| 8897  | Headsets                     |
| 20047 | Mice, Trackballs & Touchpads |
| 24858 | Power Tool Sets              |
| 35894 | Lenses                       |
| 11266 | Internal Hard Disk Drives    |
| 34353 | PC Laptops & Netbooks        |
| 33568 | Cell Phones & Smartphones    |

|       | review title \                                 |
|-------|------------------------------------------------|
| 37765 |                                                |
| 1561  | Fun time                                       |
| 20403 | Awesome!                                       |
| 40541 | Ok                                             |
| 31419 | It doesn't have a user manual and bi ear piece |
| 8897  | Air Pods                                       |
| 20047 | Happy                                          |
| 24858 | Great drill set!                               |
| 35894 | Not impressed                                  |
| 11266 | POST                                           |
| 34353 | Exclelent                                      |
| 33568 | Fuck                                           |

|       | review content                                    | rating \ |
|-------|---------------------------------------------------|----------|
| 37765 | ,                                                 | 5        |
| 1561  | Been wanting this for awhile lots of fun          | 5        |
| 20403 | Great product for the price!                      | 5        |
| 40541 | Verified purchase: Yes   Condition: pre-owned...  | 5        |
| 31419 | It's not fully charged battery                    | 3        |
| 8897  | I do a lot of meetings from home now and they ... | 4        |
| 20047 | Happy with this product                           | 5        |
| 24858 | The build quality is really, really good. The...  | 5        |
| 35894 | Disappointed in the focusing. I knew it was m...  | 2        |
| 11266 | Good afternoon. Thank you, I received the parc... | 5        |
| 34353 | Good product!!!                                   | 5        |
| 33568 | Now thats cheap.                                  | 5        |

|       | interest_category            | Predicted                 |
|-------|------------------------------|---------------------------|
| 37765 | Cell Phones & Smartphones    | PC Laptops & Netbooks     |
| 1561  | Video Game Consoles          | Video Games               |
| 20403 | Mice, Trackballs & Touchpads | Monitors                  |
| 40541 | Headphones                   | Cell Phones & Smartphones |
| 31419 | Cell Phones & Smartphones    | Headphones                |
| 8897  | Headsets                     | Headphones                |
| 20047 | Mice, Trackballs & Touchpads | Cell Phones & Smartphones |
| 24858 | Power Tool Sets              | Cordless Drills           |
| 35894 | Lenses                       | Digital Cameras           |
| 11266 | Internal Hard Disk Drives    | Smart Speakers            |
| 34353 | PC Laptops & Netbooks        | Cell Phones & Smartphones |

## 28 29. Interpreting the Four Models

### 28.1 Logistic Regression

Strengths: - strong baseline for high-dimensional TF-IDF; - fast inference; - directly interpretable coefficients; - native class probabilities.

Weakness: - decision boundaries are linear in the selected TF-IDF feature space.

### 28.2 Linear SVM

Strengths: - efficient for large sparse text representations; - strong large-margin classifier; - often competitive with Logistic Regression on text.

Weakness: - `LinearSVC` does not provide calibrated probabilities directly.

### 28.3 Multinomial Naive Bayes

Strengths: - very fast; - simple; - effective baseline for token-frequency-like representations.

Weakness: - relies on the conditional-independence assumptions of Naive Bayes.

### 28.4 Deep Learning - NumPy

Strengths: - learns an internal text representation through the embedding matrix; - captures non-linear transformations through multiple ReLU layers; - follows the complete From-Scratch learning cycle: `Forward` → `Loss` → `Backpropagation` → `Gradient Descent`.

Weakness: - manual NumPy training is much less optimized than production Deep Learning frameworks; - training quality depends on vocabulary, sequence length, architecture, learning rate, and number of epochs; - the mean-pooling design loses word-order information after the embedding stage.

## 29 30. Final Model Ranking

The table below is the single four-model result that should be used for the report.

The model is ranked by Macro F1, but Accuracy, Recall, ROC-AUC, and training cost remain visible.

A deployment decision should consider both predictive quality and operational constraints.

```
[54]: final_four_ranked = final_four.copy()
      final_four_ranked.insert(0, "Rank", np.arange(1, len(final_four_ranked) + 1))

      display(final_four_ranked)
```

|   | Rank | Model               | Accuracy | Precision_macro | Recall_macro | \ |
|---|------|---------------------|----------|-----------------|--------------|---|
| 0 | 1    | Linear SVM          | 0.776193 | 0.781943        | 0.629196     |   |
| 1 | 2    | Logistic Regression | 0.771268 | 0.854291        | 0.596947     |   |

|   |   |                         |          |          |          |
|---|---|-------------------------|----------|----------|----------|
| 2 | 3 | Deep Learning - NumPy   | 0.688232 | 0.554260 | 0.562831 |
| 3 | 4 | Multinomial Naive Bayes | 0.688232 | 0.888586 | 0.391919 |

|   | F1_macro | Balanced_Accuracy | ROC_AUC_OVR_macro | Training_Time_sec |
|---|----------|-------------------|-------------------|-------------------|
| 0 | 0.690231 | 0.629196          | 0.944697          | 10.740268         |
| 1 | 0.686306 | 0.596947          | 0.957819          | 7.488329          |
| 2 | 0.555029 | 0.562831          | 0.939520          | NaN               |
| 3 | 0.483426 | 0.391919          | 0.914931          | 0.050471          |

## 30 31. A3-Style Deep Learning Checklist

### 30.0.1 Data preparation

Clean review text

↓

Train-only vocabulary

↓

Tokens

↓

Token IDs

↓

Fixed-length sequences

### 30.0.2 Network

Token IDs

→ Embedding

→ Mean Pooling

→ Dense

→ ReLU

→ Dense

→ ReLU

→ Softmax

### 30.0.3 Learning

Step 3 Forward Propagation

→ Step 4 Cross-Entropy Loss

→ Step 5 Backpropagation

→ Step 6 Gradient Descent

→ Step 7 Training Loop

### 30.0.4 Evaluation

Step 8 Prediction

→ Step 9 Evaluation

→ Accuracy

→ Macro Precision

→ Macro Recall

- Macro F1
- Balanced Accuracy
- ROC-AUC
- Confusion Matrix

### 30.0.5 Visualization

Category distribution  
 Rating distribution  
 Text length distributions  
 Feature-selection scores  
 Traditional ML comparison  
 Best ML confusion matrix  
 DL training/validation loss  
 DL confusion matrix  
 Four-model comparison  
 Four-model heatmap  
 Confidence distribution  
 Error analysis

## 31 32. Important Questions and Answers

### 31.0.1 1. Why is this a classification problem?

Because the target is a discrete product-interest category rather than a continuous number.

### 31.0.2 2. Why is category not included in X?

Because category is the target. Including it in the input would be direct target leakage.

### 31.0.3 3. Why are title and content combined?

The title contains concise product cues while the content contains detailed descriptions, preferences, and experience. Combining them preserves both information sources.

### 31.0.4 4. Why is rating retained?

The rating is a compact auxiliary signal about the customer's review experience. It is not the primary representation, so the text remains the dominant input.

### 31.0.5 5. Why use TF-IDF for ML?

TF-IDF converts text into sparse numerical features and reduces the influence of very common terms while emphasizing terms that help distinguish documents.

### 31.0.6 6. Why use feature selection?

A large vocabulary can contain redundant, rare, or low-information dimensions. Chi-square feature selection keeps a controlled subset with stronger association to the training target.

### **31.0.7 7. How is missing review title handled?**

A missing title is replaced by an empty string because title is optional. Review content is used as the primary text signal.

### **31.0.8 8. How are exact duplicates handled?**

Exact duplicate rows are removed before data splitting so the same observation is not given extra influence.

### **31.0.9 9. Why group rare categories into Other?**

The raw category field is strongly long-tailed. A controlled target with enough examples per class makes model training and the from-scratch neural network more stable while preserving all observations through the `Other` category.

### **31.0.10 10. Why must the vectorizer be fit only on the training data?**

Because fitting it on test data would allow information from the test distribution to influence the representation and create data leakage.

### **31.0.11 11. Why does the Deep Learning model use embeddings?**

A token ID such as 1523 is only an index. The embedding matrix turns that discrete symbol into a learned dense vector that can capture useful relationships during training.

### **31.0.12 12. Why use Softmax instead of Sigmoid?**

The application predicts one category among multiple mutually exclusive interest classes. Softmax produces a probability distribution across those classes.

### **31.0.13 13. What does backpropagation calculate?**

It calculates the gradient of the loss with respect to all trainable parameters by repeatedly applying the chain rule backward through the network.

### **31.0.14 14. What does gradient descent do?**

It moves parameters in the direction that reduces the loss:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

### **31.0.15 15. Are the four models compared fairly?**

Yes, because all four models use the same train/validation/test split and are evaluated on the same untouched test set with the same core metrics.

### **31.0.16 16. Which model should be selected?**

The notebook uses Macro F1 as the primary ranking metric, then checks the full metric profile, confusion matrix, training cost, and deployment requirements before final selection.

## 32 33. Deployment Demonstration

Assignment 02 requires the deployed application to follow:

```
User Input
  ↓
Validation
  ↓
Preprocessing
  ↓
Saved Model
  ↓
Prediction
  ↓
Result
```

For the Comments application, a Web/Mobile input can contain:

- Review title
- Review content
- Rating

The deployment result should show **both**:

1. the selected best traditional ML prediction;
2. the Deep Learning prediction.

Where probability output exists, the interface should show: - predicted class; - confidence/probability; - top competing classes; - model metrics for context.

The notebook therefore creates one shared inference example below.

```
[55]: best_ml_deploy_name = best_ml_name

demo_input = pd.DataFrame([
    "review title": "Excellent wireless headset",
    "review content": (
        "Very comfortable for long gaming sessions. "
        "The sound is clear, the microphone works well, and setup was easy."
    ),
    "rating": 5,
])

display(demo_input)
```

```
      review title \
0  Excellent wireless headset

      review content  rating
0  Very comfortable for long gaming sessions. The...      5
```

### 32.0.1 ML Deployment Inference Helper

This helper applies the **same saved preprocessing representation** used at training:

- title + content
- saved TF-IDF vectorizer
- saved feature selector
- rating representation
- saved ML model

It does not fit anything on user input.

```
[56]: def transform_ml_input(input_df):
    data = input_df.copy()

    data["review title"] = data["review title"].fillna("").astype(str)
    data["review content"] = data["review content"].fillna("").astype(str)
    data["rating"] = pd.to_numeric(data["rating"], errors="coerce").fillna(5).
    ↪astype(int)

    title_clean = data["review title"].map(normalize_review_text)
    content_clean = data["review content"].map(normalize_review_text)

    combined = (
        title_clean.str.strip()
        + " [SEP] "
        + content_clean.str.strip()
    ).str.strip()

    tfidf = vectorizer.transform(combined.tolist())
    tfidf_selected = selector.transform(tfidf)

    rating_aux = (data["rating"].to_numpy(dtype=np.float64) - 1.0) / 4.0

    return hstack(
        [tfidf_selected, csr_matrix(rating_aux.reshape(-1, 1))],
        format="csr",
    )

X_demo_ml = transform_ml_input(demo_input)

ml_demo_model = ml_models[best_ml_deploy_name]
ml_demo_pred = ml_demo_model.predict(X_demo_ml)

if best_ml_deploy_name == "Linear SVM":
    ml_demo_scores = ml_demo_model.decision_function(X_demo_ml)
    ml_demo_prob = softmax_numpy(ml_demo_scores)
    ml_prob_note = "Normalized decision scores; not calibrated probabilities."
else:
```

```

ml_demo_prob = ml_demo_model.predict_proba(X_demo_ml)
ml_prob_note = "Model probability output."

ml_demo_id = int(ml_demo_pred[0])
ml_demo_top = np.argsort(ml_demo_prob[0])[:, :-1][:5]

ml_demo_table = pd.DataFrame({
    "Rank": np.arange(1, 6),
    "Interest category": [ID_TO_CLASS[i] for i in ml_demo_top],
    "Score": [ml_demo_prob[0, i] for i in ml_demo_top],
})

print("Best ML model:", best_ml_deploy_name)
print("Prediction:", ID_TO_CLASS[ml_demo_id])
print(ml_prob_note)
display(ml_demo_table)

```

Best ML model: Linear SVM

Prediction: Headphones

Normalized decision scores; not calibrated probabilities.

|   | Rank | Interest category   | Score    |
|---|------|---------------------|----------|
| 0 | 1    | Headphones          | 0.243626 |
| 1 | 2    | Keyboards & Keypads | 0.042319 |
| 2 | 3    | Headsets            | 0.041923 |
| 3 | 4    | Video Game Consoles | 0.037369 |
| 4 | 5    | Wireless Routers    | 0.036633 |

### 32.0.2 Deep Learning Deployment Inference Helper

The Deep Learning path uses the saved vocabulary:

```

review text
→ tokens
→ token IDs
→ embedding
→ neural network
→ Softmax probabilities

```

The vocabulary and network parameters are learned from the training data and must be reused unchanged during inference.

```

[57]: def transform_dl_input(input_df):
    data = input_df.copy()

    data["review title"] = data["review title"].fillna("").astype(str)
    data["review content"] = data["review content"].fillna("").astype(str)

    title_clean = data["review title"].map(normalize_review_text)

```

```

content_clean = data["review content"].map(normalize_review_text)

combined = (
    title_clean.str.strip()
    + " [SEP] "
    + content_clean.str.strip()
).str.strip()

return np.stack([encode_text(x) for x in combined.tolist()])

X_demo_dl = transform_dl_input(demo_input)
dl_demo_prob = dl_model.predict_proba(X_demo_dl)
dl_demo_pred = np.argmax(dl_demo_prob, axis=1)

dl_demo_top = np.argsort(dl_demo_prob[0])[:, :-1][:5]

dl_demo_table = pd.DataFrame({
    "Rank": np.arange(1, 6),
    "Interest category": [ID_TO_CLASS[i] for i in dl_demo_top],
    "Probability": [dl_demo_prob[0, i] for i in dl_demo_top],
})

print("Deep Learning prediction:", ID_TO_CLASS[int(dl_demo_pred[0])])
display(dl_demo_table)

```

Deep Learning prediction: Headphones

|   | Rank | Interest category         | Probability  |
|---|------|---------------------------|--------------|
| 0 | 1    | Headphones                | 9.999826e-01 |
| 1 | 2    | Headsets                  | 1.499925e-05 |
| 2 | 3    | Cell Phones & Smartphones | 1.906718e-06 |
| 3 | 4    | Monitors                  | 5.066414e-07 |
| 4 | 5    | Camcorders                | 2.508773e-09 |

```

[58]: # Combined Web/Mobile-style result.
combined_demo = pd.DataFrame({
    "Model": [
        f"Best ML - {best_ml_deploy_name}",
        "Deep Learning - NumPy",
    ],
    "Prediction": [
        ID_TO_CLASS[ml_demo_id],
        ID_TO_CLASS[int(dl_demo_pred[0])],
    ],
    "Top Confidence": [
        float(np.max(ml_demo_prob[0])),
        float(np.max(dl_demo_prob[0])),
    ],
})

```

```

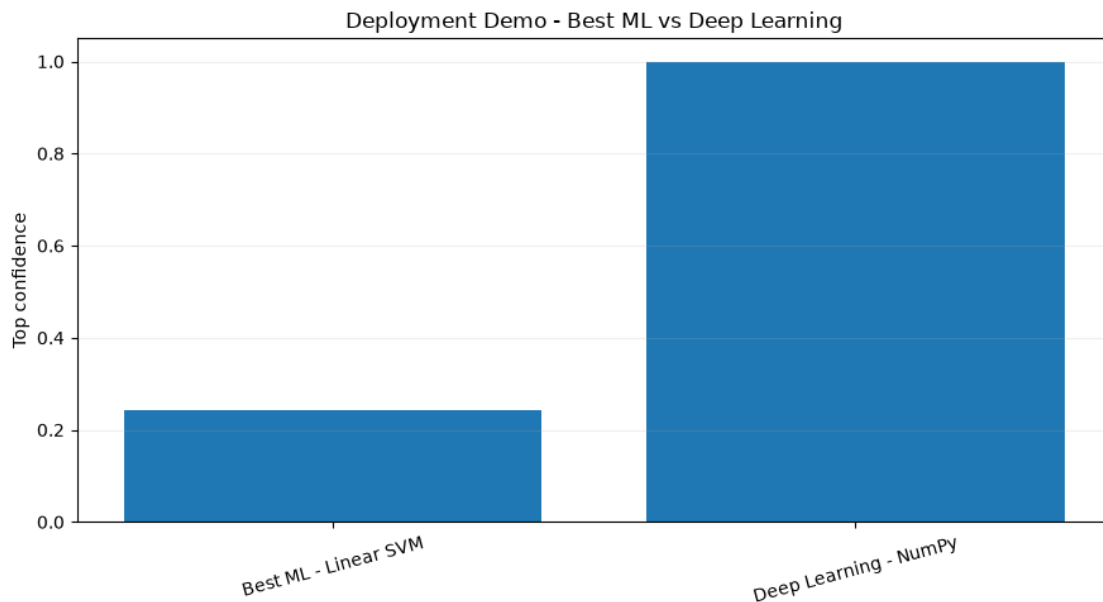
})

display(combined_demo)

plt.figure(figsize=(9, 5))
plt.bar(
    combined_demo["Model"],
    combined_demo["Top Confidence"],
)
plt.ylim(0, 1.05)
plt.ylabel("Top confidence")
plt.title("Deployment Demo - Best ML vs Deep Learning")
plt.xticks(rotation=15)
plt.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.savefig(FIG_DIR / "19_deployment_two_model_confidence.png", dpi=180)
plt.show()

```

|   | Model                 | Prediction | Top Confidence |
|---|-----------------------|------------|----------------|
| 0 | Best ML - Linear SVM  | Headphones | 0.243626       |
| 1 | Deep Learning - NumPy | Headphones | 0.999983       |



### 32.0.3 Deployment Visualization Payload

A Web/Mobile result screen can be built from three reusable pieces:

Prediction Card

→ predicted interest category

Probability / Confidence Chart  
→ top 3-5 candidate interests

Model Comparison Panel  
→ Best ML metrics vs Deep Learning metrics

The next cell creates the compact table that a backend can expose to the UI.

```
[59]: deployment_summary = {
    "best_ml_model": best_ml_deploy_name,
    "best_ml_prediction": ID_TO_CLASS[ml_demo_id],
    "best_ml_top5": [
        {
            "category": ID_TO_CLASS[int(i)],
            "score": float(ml_demo_prob[0, i]),
        }
        for i in ml_demo_top
    ],
    "deep_learning_prediction": ID_TO_CLASS[int(dl_demo_pred[0])],
    "deep_learning_top5": [
        {
            "category": ID_TO_CLASS[int(i)],
            "probability": float(dl_demo_prob[0, i]),
        }
        for i in dl_demo_top
    ],
}

display(pd.DataFrame([
    {
        "Model": "Best ML",
        "Prediction": deployment_summary["best_ml_prediction"],
        "Top Score": deployment_summary["best_ml_top5"][0]["score"],
    },
    {
        "Model": "Deep Learning",
        "Prediction": deployment_summary["deep_learning_prediction"],
        "Top Score": deployment_summary["deep_learning_top5"][0]["probability"],
    },
]))
```

|   | Model         | Prediction | Top Score |
|---|---------------|------------|-----------|
| 0 | Best ML       | Headphones | 0.243626  |
| 1 | Deep Learning | Headphones | 0.999983  |

## 33 34. Persist the Best ML Model and Deep Learning Model

The deployed application must not retrain models.

We therefore save:

### 33.0.1 Traditional ML

- TF-IDF vectorizer
- chi-square feature selector
- best ML model
- class mapping
- rating representation

### 33.0.2 Deep Learning

- NumPy weights
- vocabulary/tokenizer information
- architecture configuration
- class mapping

The same learned preprocessing representation must be reused during Web/Mobile inference.

```
[60]: # Save best ML bundle.
best_ml_bundle = {
    "model_name": best_ml_deploy_name,
    "model": ml_models[best_ml_deploy_name],
    "vectorizer": vectorizer,
    "selector": selector,
    "class_order": CLASS_ORDER,
    "class_to_id": CLASS_TO_ID,
    "selected_tfidf_features": int(SELECTED_TFIDF_FEATURES),
    "top_k_categories": int(TOP_K),
    "rating_representation": "minmax 1-5 -> [0, 1]",
}

BEST_ML_BUNDLE_PATH = MODEL_DIR / "comments_best_ml_bundle.joblib"
joblib.dump(best_ml_bundle, BEST_ML_BUNDLE_PATH)

# Save Deep Learning weights and metadata.
DL_WEIGHTS_PATH = MODEL_DIR / "comments_dl_weights.npz"
np.savez(
    DL_WEIGHTS_PATH,
    embedding=dl_model.embedding,
    W1=dl_model.W1,
    b1=dl_model.b1,
    W2=dl_model.W2,
    b2=dl_model.b2,
    W3=dl_model.W3,
```

```

        b3=dl_model.b3,
    )

DL_CONFIG_PATH = MODEL_DIR / "comments_dl_config.json"
dl_config = {
    "task": "ecommerce_product_interest_text_classification",
    "architecture": {
        "vocab_size": len(token_to_id),
        "embedding_dim": DL_EMBED_DIM,
        "hidden1": DL_HIDDEN_1,
        "hidden2": DL_HIDDEN_2,
        "output_dim": DL_OUTPUT_DIM,
        "max_len": MAX_LEN,
        "padding_id": PAD_ID,
        "unknown_id": UNK_ID,
    },
    "learning_rate": DL_LEARNING_RATE,
    "epochs": DL_EPOCHS,
    "batch_size": DL_BATCH_SIZE,
    "deep_learning_framework": "None - NumPy only",
    "class_order": CLASS_ORDER,
}

DL_VOCAB_PATH = MODEL_DIR / "comments_dl_vocab.json"
json.dump(
    {
        "token_to_id": token_to_id,
        "id_to_token": id_to_token,
    },
    open(DL_VOCAB_PATH, "w", encoding="utf-8"),
    ensure_ascii=False,
    indent=2,
)

json.dump(
    dl_config,
    open(DL_CONFIG_PATH, "w", encoding="utf-8"),
    ensure_ascii=False,
    indent=2,
)

print("Saved:")
print(BEST_ML_BUNDLE_PATH)
print(DL_WEIGHTS_PATH)
print(DL_VOCAB_PATH)
print(DL_CONFIG_PATH)

```

Saved:

```

C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\comments\model_comments\comments_best_ml_bundle.joblib
C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\comments\model_comments\comments_dl_weights.npz
C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\comments\model_comments\comments_dl_vocab.json
C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\comments\model_comments\comments_dl_config.json

```

## 34 35. Save Final Comparison and Deployment Metadata

The following artifacts support the report and the Web/Mobile application.

```

model_comments/
  comments_best_ml_bundle.joblib
  comments_dl_weights.npz
  comments_dl_vocab.json
  comments_dl_config.json
  comments_four_model_comparison.csv
  comments_deployment_metrics.json
  figures/

```

```

[61]: comparison_path = MODEL_DIR / "comments_four_model_comparison.csv"
      final_four_ranked.to_csv(comparison_path, index=False)

      deployment_metrics = {
          "task": "multi_class_ecommerce_product_interest_text_classification",
          "classes": CLASS_ORDER,
          "best_ml_model": best_ml_deploy_name,
          "best_ml_metrics": {
              k: (None if pd.isna(v) else float(v))
              for k, v in final_four.set_index("Model").loc[
                  best_ml_deploy_name
              ].to_dict().items()
              if k != "Training_Time_sec"
          },
          "deep_learning_metrics": {
              k: (
                  None if pd.isna(v)
                  else float(v)
              )
              for k, v in final_four.set_index("Model").loc[
                  "Deep Learning - NumPy"
              ].to_dict().items()
              if k != "Training_Time_sec"
          },
          "framework": "Traditional ML: scikit-learn; Deep Learning: NumPy only",
          "split": "70% train / 15% validation / 15% test",

```

```

        "random_seed": SEED,
    }

    deployment_metrics_path = MODEL_DIR / "comments_deployment_metrics.json"
    json.dump(
        deployment_metrics,
        open(deployment_metrics_path, "w", encoding="utf-8"),
        ensure_ascii=False,
        indent=2,
    )

    print("Comparison CSV:", comparison_path)
    print("Deployment JSON:", deployment_metrics_path)

```

Comparison CSV: C:\Users\ADMIN\Downloads\Intelligent System\assignment  
 3\DATA\comments\model\_comments\comments\_four\_model\_comparison.csv  
 Deployment JSON: C:\Users\ADMIN\Downloads\Intelligent System\assignment  
 3\DATA\comments\model\_comments\comments\_deployment\_metrics.json

## 35 36. Reproducibility Check

A second user should be able to reproduce the core experiment from the documented settings.

Important reproducibility information:

- random seed = 42
- split = 70/15/15, stratified
- target = top 15 raw categories + Other
- TF-IDF word/bigram representation
- chi-square selection = up to 20,000 text features
- rating appended as a compact auxiliary signal
- NumPy vocabulary learned from training reviews only
- maximum DL sequence length = 60
- embedding dimension = 24
- hidden layers = 64 → 32
- output = number of interest classes
- Deep Learning optimizer = manual gradient descent
- no TensorFlow / PyTorch / Keras

```

[62]: reproducibility = {
    "random_seed": SEED,
    "split": "70% train / 15% validation / 15% test",
    "stratified": True,
    "target": "top 30 raw categories",
    "raw_rows": int(len(df_raw)),
    "clean_rows": int(len(df)),
    "raw_categories": int(df["category"].nunique()),
    "modeling_classes": CLASS_ORDER,

```

```

"tfidf_ngram_range": [1, 2],
"tfidf_min_df": TFIDF_MIN_DF,
"tfidf_max_df": TFIDF_MAX_DF,
"tfidf_max_features": TFIDF_MAX_FEATURES,
"selected_text_features": int(SELECTED_TFIDF_FEATURES),
"dl_vocab_size": int(len(token_to_id)),
"dl_max_len": MAX_LEN,
"dl_embedding_dim": DL_EMBED_DIM,
"dl_hidden1": DL_HIDDEN_1,
"dl_hidden2": DL_HIDDEN_2,
"dl_learning_rate": DL_LEARNING_RATE,
"dl_epochs": DL_EPOCHS,
"dl_batch_size": DL_BATCH_SIZE,
"deep_learning_framework": "None - NumPy only",
"best_ml_model": best_ml_deploy_name,
}

repro_path = MODEL_DIR / "comments_reproducibility.json"
json.dump(reproducibility, open(repro_path, "w", encoding="utf-8"), indent=2)

display(pd.DataFrame(
    list(reproducibility.items()),
    columns=["Parameter", "Value"],
))

```

|    | Parameter               | Value                                             |
|----|-------------------------|---------------------------------------------------|
| 0  | random_seed             | 42                                                |
| 1  | split                   | 70% train / 15% validation / 15% test             |
| 2  | stratified              | True                                              |
| 3  | target                  | top 30 raw categories                             |
| 4  | raw_rows                | 44756                                             |
| 5  | clean_rows              | 39255                                             |
| 6  | raw_categories          | 30                                                |
| 7  | modeling_classes        | [Cell Phones & Smartphones, Digital Cameras, V... |
| 8  | tfidf_ngram_range       | [1, 2]                                            |
| 9  | tfidf_min_df            | 2                                                 |
| 10 | tfidf_max_df            | 0.98                                              |
| 11 | tfidf_max_features      | 30000                                             |
| 12 | selected_text_features  | 12000                                             |
| 13 | dl_vocab_size           | 4000                                              |
| 14 | dl_max_len              | 40                                                |
| 15 | dl_embedding_dim        | 64                                                |
| 16 | dl_hidden1              | 256                                               |
| 17 | dl_hidden2              | 128                                               |
| 18 | dl_learning_rate        | 0.001                                             |
| 19 | dl_epochs               | 100                                               |
| 20 | dl_batch_size           | 256                                               |
| 21 | deep_learning_framework | None - NumPy only                                 |

## 36 37. Final Conclusion

The final conclusion should be written from the executed results rather than from a fixed template.

The experiment demonstrates the complete Assignment 02 principle:

```
Raw e-commerce reviews
→ Clean
→ Text representation
→ Feature engineering / feature selection
→ 3 traditional ML models
→ Deep Learning representation learning
→ Fair four-model evaluation
→ Persist
→ Deploy
```

### 36.0.1 Main lessons

1. Text data cannot be given directly to a mathematical model; it must first be represented.
2. Feature engineering remains critical even when the raw dataset has only a few columns.
3. For ML, TF-IDF plus feature selection creates a controlled sparse numerical representation.
4. For Deep Learning, token IDs are mapped to learned embeddings and transformed by non-linear layers.
5. Backpropagation remains the chain-rule mechanism described in the Deep Learning lecture.
6. The final comparison must include all four models together on one untouched test set.
7. A deployable intelligent system includes preprocessing, saved models, inference logic, and user-facing prediction results—not only a training script.

### 36.1 Final system summary

```
User
↓
Review title + review content + rating
↓
Validation
↓
Saved preprocessing
  Best ML: TF-IDF → feature selection → classifier
  Deep Learning: tokenizer → token IDs → embedding → NumPy MLP
↓
Prediction
↓
Interest category + confidence/probabilities
↓
Web / Mobile UI
```

The notebook therefore covers the complete path from **Data** → **Representation** → **Learning**

→ **Evaluation** → **Deployment**, with the Deep Learning implementation shown explicitly step by step.

## 36.2 Web

